



PROJET PROPEL

Rapport fonctionnel

REALISER PAR

Bilal Magguilej
Nourelhouda Ouhba
Mouhssine El Haouary
Mohssine El Addaoui
Salman El-Ghrich
Somia Jdia

ENCADRER PAR

M.GHAILANI



+212 602-115105



<https://github.com/BilalMagguilej/CV-Generator>

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réussite de ce projet d'intégration.

Nous remercions chaleureusement **M. GHAILANI**, professeur responsable du module, pour son encadrement, ses orientations précieuses et son suivi rigoureux tout au long du semestre. Ses conseils nous ont permis de maintenir le cap et de structurer notre travail de manière professionnelle.

Nous adressons également nos sincères remerciements aux **élèves ingénieurs du niveau supérieur** ainsi qu'aux **lauréats** qui ont partagé leurs retours d'expérience et leurs recommandations, contribuant ainsi à l'amélioration continue de notre projet.

Nous tenons à exprimer notre reconnaissance aux **membres du jury** pour l'attention qu'ils porteront à l'évaluation de notre travail et pour le temps qu'ils nous accorderont lors de la soutenance.

Enfin, nous remercions tous ceux qui, de près ou de loin, ont contribué à la réalisation de ce projet par leur soutien, leurs encouragements ou leur expertise.

L'équipe CV_Generator

Résumé

Ce projet présente la conception et le développement de **Propel**, une plateforme web intelligente de génération de CV assistée par intelligence artificielle. L'application permet aux utilisateurs de générer un curriculum vitae personnalisé et optimisé pour une offre d'emploi spécifique en quelques secondes. Le système repose sur une architecture en microservices combinant Angular pour le frontend, ASP.NET Core pour les services backend, et une pipeline d'agents IA en Python utilisant LangChain et LangGraph pour l'extraction, l'analyse, l'optimisation et la livraison des CV. L'infrastructure est entièrement conteneurisée avec Docker et déployée via un pipeline CI/CD complet intégrant des outils de sécurité et de qualité (SonarCloud, Trivy, OWASP ZAP). Le projet a été réalisé par une équipe de six développeurs fullstack selon une méthodologie SCRUM adaptée.

Mots-clés : Génération de CV, Intelligence Artificielle, Microservices, Angular, ASP.NET Core, LangChain, Docker, DevSecOps.

Abstract

This project presents the design and development of **Propel**, an intelligent web platform for AI-powered CV generation. The application enables users to generate a tailored and optimized curriculum vitae for a specific job offer in seconds. The system is built on a microservices architecture combining Angular for the frontend, ASP.NET Core for backend services, and a Python AI agent pipeline using LangChain and LangGraph for extraction, analysis, optimization, and delivery of CVs. The infrastructure is fully containerized with Docker and deployed through a complete CI/CD pipeline integrating security and quality tools (SonarCloud, Trivy, OWASP ZAP). The project was carried out by a team of six full-stack developers following an adapted SCRUM methodology.

Keywords : CV Generation, Artificial Intelligence, Microservices, Angular, ASP.NET Core, LangChain, Docker, DevSecOps.

Liste des acronymes

.NET Microsoft .NET Framework. [iv](#), [38](#)

ALB Application Load Balancer. [iv](#), [32](#)

Angular Angular Framework. [iv](#)

API Application Programming Interface. [iv](#), [39](#)

ASG Auto Scaling Group. [iv](#), [33](#)

ATS Applicant Tracking System. [iv](#), [46](#)

CD Continuous Deployment. [iv](#)

CI Continuous Integration. [iv](#)

CRUD Create, Read, Update, Delete. [iv](#), [37](#), [39](#), [43](#)

DAST Dynamic Application Security Testing. [iv](#)

E2E End-to-End. [iv](#)

gRPC gRPC Remote Procedure Call. [iv](#), [37](#), [38](#), [42](#), [45](#)

HTTP HyperText Transfer Protocol. [iv](#), [41](#)

HTTPS HyperText Transfer Protocol Secure. [iv](#)

IaC Infrastructure as Code. [iv](#), [32](#)

JSON JavaScript Object Notation. [iv](#)

JWT JSON Web Token. [iv](#), [36](#)

Kafka Apache Kafka. [iv](#), [36–38](#), [41](#), [42](#), [44](#), [46](#)

LLM Large Language Model. [iv](#)

OAuth Open Authorization. [iv](#), [46](#)

OIDC OpenID Connect. [iv](#), [46](#)

OWASP Open Web Application Security Project. [iv](#)

PDF Portable Document Format. [iv](#), [40](#), [46](#)

PR Pull Request. [iv](#)

RAG Retrieval-Augmented Generation. [iv](#), [33](#), [41](#)

REST Representational State Transfer. [iv](#), [37](#), [39](#), [43](#)

S3 Simple Storage Service. [iv](#), [46](#)

SOLID Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion. [iv](#)

SPA Single Page Application. [iv](#), [35](#)

SQL Structured Query Language. [iv](#)

SSO Single Sign-On. [iv](#)

VM Virtual Machine. [iv](#)

VPC Virtual Private Cloud. [iv](#), [32](#)

XSS Cross-Site Scripting. [iv](#), [36](#)

YARP Yet Another Reverse Proxy. [iv](#), [36](#)

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
Liste des acronymes	iv
1 Introduction générale	1
2 Contexte général du projet	2
2.1 Introduction	2
2.2 Présentation du projet	2
2.2.1 Cadre du projet	2
2.2.2 Objectifs du projet	2
2.2.3 Méthodologie de gestion de projet	3
2.2.4 Présentation de l'équipe	3
2.2.5 Planification du projet	5
2.3 Conclusion	10
3 Analyse et conception	11
3.1 Introduction	11
3.2 Étude de l'existant	11
3.3 Étude des besoins fonctionnels	12
3.4 Étude des besoins non fonctionnels	13
3.5 Identification des acteurs	13
3.6 Modélisation du système	14
3.6.1 Diagramme de cas d'utilisation	14
3.6.2 Diagramme de classes	18
3.6.3 Diagramme de séquence — Flux d'authentification	19
3.6.4 Modèle relationnel de la base de données	20
3.7 Conclusion	27

4	Réalisation	28
4.1	Introduction	28
4.2	Étude technique	28
4.2.1	Architecture physique	28
4.2.2	Architecture logicielle	34
4.2.3	Présentation des outils utilisés	46
4.3	Cycle DevSecOps	47
4.3.1	Présentation et importance dans le projet	47
4.3.2	Outils DevSecOps utilisés	48
4.3.3	Application du cycle DevSecOps dans le projet	48
4.3.4	Sécurité applicative	49
4.4	Gestion du code source	49
4.4.1	Utilisation de Git	49
4.4.2	Organisation des branches	50
4.4.3	Gestion des commits et collaboration	50
4.5	Implémentation	50
4.5.1	Développement Backend	50
4.5.2	Pipeline de génération de CV	52
4.5.3	Développement Frontend	53
4.5.4	Base de données	53
4.5.5	Documentation des API	53
4.6	Déploiement	54
4.6.1	Mise en place de l'environnement de développement	54
4.6.2	Dockerisation de l'application	55
4.6.3	Pipeline CI/CD	55
4.7	Présentation de l'application	56
4.7.1	Interface utilisateur	56
4.7.2	Fonctionnalités principales	58
4.7.3	Démonstrations du fonctionnement	58
4.8	Conclusion	60
5	Conclusion générale et perspectives	61
5.1	Conclusion générale	61
5.2	Perspectives	61

Table des figures

2.1	Présentation de l'équipe CV_Generator	4
2.2	Organigramme fonctionnel de l'équipe	5
2.3	Diagramme de Gantt réel	6
3.1	Cas d'utilisation — Authentification	14
3.2	Cas d'utilisation — Gestion du profil	15
3.3	Cas d'utilisation — Génération de CV	16
3.4	Cas d'utilisation — Suivi des candidatures	17
3.5	Cas d'utilisation — Offres d'emploi et notifications	18
3.6	Diagramme de classes — Relations inter-services	18
3.7	Diagramme de séquence — Flux d'authentification OIDC	19
3.8	Modèle relationnel — Base Utilisateurs	21
3.9	Modèle relationnel — Base Contenu Utilisateur	22
3.10	Modèle relationnel — Base Candidatures	23
3.11	Modèle relationnel — Base Workflow	24
3.12	Modèle relationnel — Base Offres d'Emploi	25
3.13	Modèle relationnel — Base CV	26
3.14	Modèle relationnel — Base Notifications	27
4.1	Architecture de déploiement locale — Répartition des VMs par membre	29
4.2	Script Terraform — Définition des ressources VMs	30
4.3	Script Terraform — Configuration réseau et déploiement	30
4.4	Résultat du provisionnement — VMs créées	31
4.5	Playbook Ansible pour la configuration des VMs	31
4.6	Logs d'exécution d'Ansible	32
4.7	Architecture logicielle du système	35
4.8	Structure du projet Frontend Angular	36
4.9	Communication inter-services — Representational State Transfer (REST), gRPC Remote Procedure Call (gRPC) et Apache Kafka (Kafka)	37
4.10	Structure du projet Backend Microsoft .NET Framework (.NET)	38
4.11	Structure du User Service	39

4.12 Structure du User Content Service	40
4.13 Structure du CV Service	41
4.14 Structure du Workflow Service	42
4.15 Structure du Application Service	43
4.16 Structure du Job Offer Service	44
4.17 Structure du Notification Service	45
4.18 Projets partagés — Common et Common Protos	45
4.19 Analyse SonarCloud réussie pour le projet	48
4.20 Images des services publiées sur DockerHub après succès du pipeline . .	49
4.21 Organisation des branches Git	50
4.22 Flux des événements Kafka — Publications et consommations	51
4.23 Topics Kafka — user.registered, user.created, trigger-live-crawl	52
4.24 Pipeline de génération de CV — De l’offre d’emploi à l’envoi par email	52
4.25 Documentation Swagger — User Service et User Content Service	53
4.26 Documentation Swagger — Application Service et Workflow Service . .	54
4.27 Documentation Swagger — CV Service et Notification Service	54
4.28 Documentation Swagger — Job Offer Service	54
4.29 Documentation Swagger — Job Offer Service (suite)	54
4.30 Conteneurs Docker en cours d’exécution	55
4.31 Pipeline CI/CD — GitHub Actions (succès)	56
4.32 Tableau de bord principal	56
4.33 Interface de gestion du portfolio et des compétences	57
4.34 Tableau Kanban de suivi des candidatures	57
4.35 Interface des agents IA pour la génération de CV	57
4.36 Page d’accueil de Propel	58
4.37 Page de connexion	59
4.38 Vue calendrier des candidatures	59
4.39 Boîte de réception des notifications	60

Liste des tableaux

2.1	Répartition des rôles au sein de l'équipe	4
3.1	Comparaison des solutions existantes	11
3.2	Besoins fonctionnels du système	12
3.3	Identification des acteurs du système	13
4.1	Outils de développement	47
4.2	Outils CI/CD et qualité	47

Chapitre 1

Introduction générale

Dans un monde professionnel de plus en plus compétitif, la qualité et la pertinence d'un curriculum vitae (CV) jouent un rôle déterminant dans le succès d'une candidature. Chaque offre d'emploi possède ses propres exigences, et un CV générique a souvent moins d'impact qu'un CV adapté spécifiquement au poste visé. Cependant, personnaliser manuellement son CV pour chaque candidature est un processus chronophage et fastidieux.

C'est dans ce contexte que s'inscrit le projet **Propel**, une plateforme web intelligente de génération de CV assistée par intelligence artificielle. L'objectif est de permettre aux candidats de générer un CV personnalisé et optimisé pour une offre d'emploi donnée en quelques secondes, tout en assurant une compatibilité optimale avec les systèmes de suivi des candidatures (ATS).

Ce rapport présente l'ensemble du travail réalisé par notre équipe dans le cadre du projet d'intégration de la 1ère année Génie Informatique, sous la supervision de M. GHAILANI.

Le rapport est structuré en trois chapitres principaux. Le premier chapitre pose le contexte général du projet, présente les objectifs, la méthodologie de gestion adoptée ainsi que l'équipe de réalisation. Le deuxième chapitre est consacré à l'analyse et la conception du système, incluant l'étude des besoins, l'identification des acteurs et la modélisation. Enfin, le troisième chapitre détaille la réalisation technique du projet, couvrant l'architecture, les outils utilisés, le cycle DevSecOps, l'implémentation et le déploiement.

Chapitre 2

Contexte général du projet

2.1 Introduction

Ce premier chapitre présente le cadre général du projet **Propel**. Nous y décrivons le contexte dans lequel s’inscrit le projet, les objectifs visés, la méthodologie de gestion adoptée, ainsi que la composition et l’organisation de l’équipe de réalisation.

2.2 Présentation du projet

2.2.1 Cadre du projet

Ce projet est réalisé dans le cadre du module **Projet d’Intégration** de la 1ère année Génie Informatique, sous la supervision de M. GHAILANI. Il constitue une mise en situation professionnelle où les étudiants sont amenés à concevoir, développer et déployer une application complète en appliquant les compétences acquises durant leur formation.

L’équipe est composée de six étudiants, chacun jouant le rôle de développeur full-stack avec une responsabilité secondaire spécifique (CI/CD, infrastructure cloud, infrastructure locale, sécurité, qualité, monitoring), reflétant ainsi une organisation DevOps complète.

2.2.2 Objectifs du projet

Les objectifs principaux du projet **Propel** sont les suivants :

- **Génération intelligente de CV** : Permettre à un utilisateur de soumettre une offre d’emploi et de recevoir un CV personnalisé et optimisé pour cette offre, généré automatiquement par une pipeline d’agents d’intelligence artificielle.
- **Suivi des candidatures** : Offrir un tableau de bord complet avec vue Kanban, liste et calendrier pour gérer l’ensemble des candidatures.

- **Gestion de portfolio** : Permettre à l'utilisateur de gérer ses projets, compétences, expériences et formations.
- **Agrégation d'offres d'emploi** : Collecter automatiquement des offres d'emploi depuis différentes plateformes via un agent de crawling.
- **Notifications temps réel** : Notifier l'utilisateur sur l'état d'avancement de ses candidatures et de ses générations de CV.
- **Déploiement industrialisé** : Mettre en place un pipeline CI/CD complet avec conteneurisation, tests automatisés, analyse de qualité et scans de sécurité.

2.2.3 Méthodologie de gestion de projet

Pour la gestion de ce projet, nous avons adopté une approche **SCRUM** adaptée à notre contexte. Le professeur joue le rôle de **Product Owner**, définissant les objectifs et validant les livrables. Le chef d'équipe assume le rôle de **SCRUM Master**, facilitant la coordination et s'assurant de l'avancement des tâches.

La méthodologie mise en œuvre se caractérise par :

- **Réunions quotidiennes** : Des sessions de coordination via **Google Meets** sont organisées régulièrement pour faire le point sur l'avancement, discuter des obstacles et planifier les tâches à venir.
- **Communication continue** : Un groupe **WhatsApp** dédié permet une communication réactive pour la résolution rapide des problèmes et le partage d'information.
- **Organisation en sprints** : Le projet est découpé en itérations (sprints) avec des objectifs clairs à chaque étape, permettant une livraison incrémentale des fonctionnalités.
- **Gestion des tâches** : La répartition des tâches est définie collectivement lors des réunions, en fonction des priorités et des compétences de chaque membre.

Cette approche agile nous a permis de nous adapter aux défis techniques rencontrés tout au long du projet et de maintenir un rythme de travail soutenu.

2.2.4 Présentation de l'équipe

L'équipe **CV_Generator** est composée de six membres, chacun occupant un rôle de développeur fullstack avec une responsabilité transverse spécifique. Cette organisation permet de couvrir l'ensemble du cycle de vie du développement logiciel, du code au déploiement en passant par la qualité et la sécurité.



FIGURE 2.1 – Présentation de l'équipe CV_Generator

TABLE 2.1 – Répartition des rôles au sein de l'équipe

Rôle	Eff.	Intervenant	Remarque
Développeur Fullstack – Chef d'équipe, CI/CD	1	Bilal Magguilej	Coordination d'équipe, pipeline CI/CD, Dockerisation complète, validation des PR
Développeur Fullstack – Infra Cloud	1	Mouhssine El Haouary	Déploiement cloud, environnement de production
Développeur Fullstack – Infra Locale	1	Somia Jdia	Terraform, provisionnement VMs locales, environnement de développement
Développeur Fullstack – Sécurité	1	Nourelhouda Ouhba	Keycloak SSO, OWASP ZAP, Trivy, authentification OAuth
Développeur Fullstack – QA	1	Mohssine El Addaoui	Tests E2E Cypress, tests de charge k6, assurance qualité
Développeur Fullstack – Monitoring	1	Salman El-Ghrich	Prometheus, Grafana, observabilité et métriques

Chaque membre a participé activement au développement des trois couches de l'application : frontend (Angular), backend (.NET) et agents IA (Python/FastAPI). Les responsabilités secondaires ont été réparties pour couvrir l'ensemble des aspects DevOps et assurer la qualité et la maintenabilité du projet.

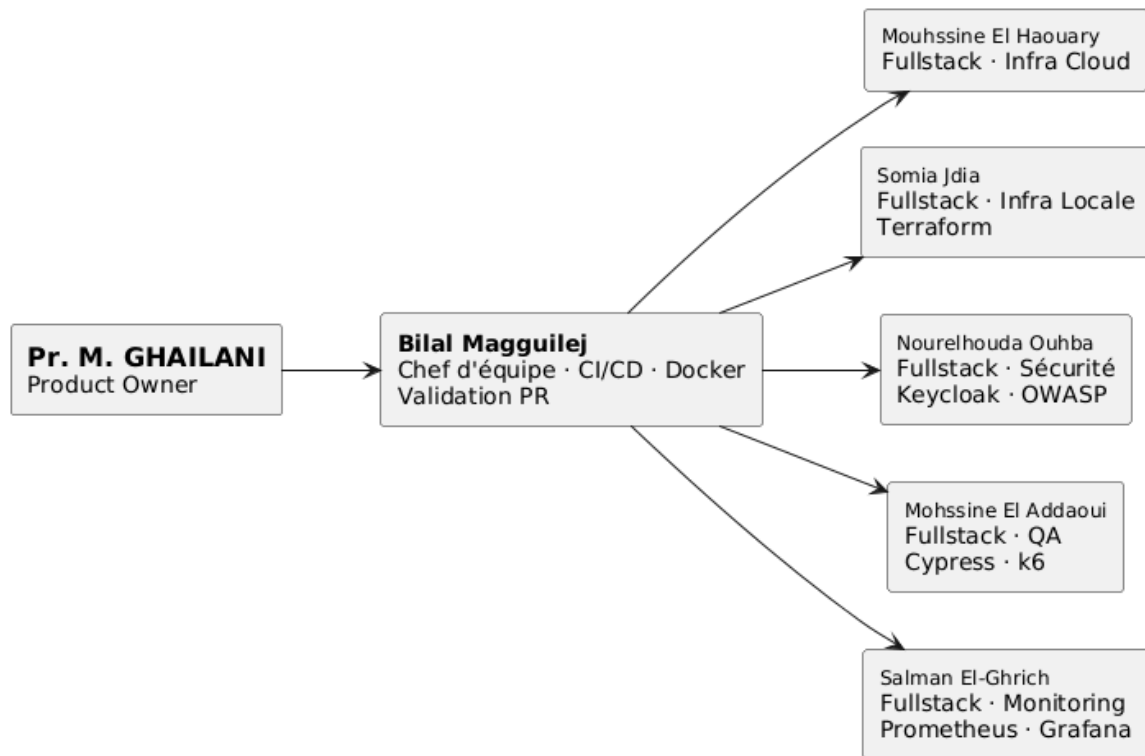


FIGURE 2.2 – Organigramme fonctionnel de l'équipe

2.2.5 Planification du projet

Le projet a été planifié sur la durée du semestre avec des jalons clés correspondant aux livrables attendus. Le diagramme de Gantt ci-dessous présente le déroulement réel du projet.

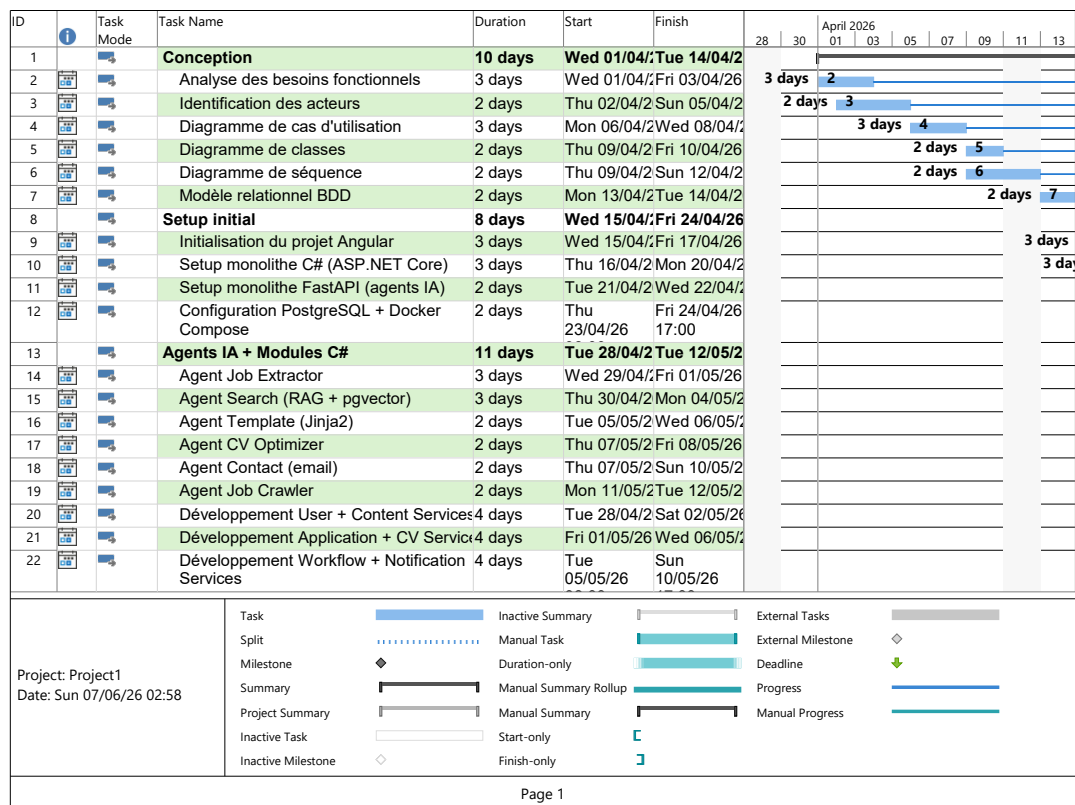


FIGURE 2.3 – Diagramme de Gantt réel

ID	Task Mode	Task Name	Duration	Start	Finish	28	30	April 2026	01	03	05	07	09	11	13
23		Développement JobOffer Service	2 days	Mon 11/05/2	Tue 12/05/2										
24		Migration microservices	8 days	Wed 13/05/2	Fri 22/05/26										
25		Découpage du monolithe C# en microservices	3 days	Wed 13/05/26	Fri 15/05/26 17:00										
26		Découpage du monolithe FastAPI en agents	2 days	Thu 14/05/26	Sun 17/05/26										
27		Configuration YARP Reverse Proxy	2 days	Mon 18/05/2	Tue 19/05/2										
28		Mise en place Kafka (KRaft)	2 days	Wed 20/05/2	Thu 21/05/2										
29		Refonte Dockerfiles multi-stage + Compose	1 day	Fri 22/05/26 08:00	Fri 22/05/26 17:00										
30		Automatisation + Tests	8 days	Wed 20/05/2	Fri 29/05/26										
31		Tests unitaires backend (xUnit)	3 days	Wed 20/05/2	Fri 22/05/26										
32		Tests unitaires frontend (Vitest)	2 days	Thu 21/05/2	Sun 24/05/2										
33		Tests unitaires agents IA (pytest)	2 days	Mon 25/05/2	Tue 26/05/2										
34		Tests E2E (Cypress)	2 days	Wed 27/05/2	Thu 28/05/2										
35		Tests de charge (k6)	2 days	Wed 27/05/2	Thu 28/05/2										
36		Analyse SonarCloud + Scan Trivy	1 day	Fri 29/05/26	Fri 29/05/26										
37		Automatisation Terraform (VMware)	3 days	Wed 20/05/2	Fri 22/05/26										
38		Intégration Frontend (Angular)	8 days	Wed 27/05/2	Fri 05/06/26										
39		Pages authentification (Keycloak OIDC)	2 days	Wed 27/05/2	Thu 28/05/2										
40		Tableau de bord + statistiques	2 days	Thu 28/05/2	Sat 30/05/26										
41		Interface génération CV	2 days	Fri 29/05/26	Mon 01/06/2										
42		Vue Kanban + Calendrier	2 days	Tue 02/06/2	Wed 03/06/2										
43		Gestion portfolio + paramètres	2 days	Thu 04/06/2	Fri 05/06/26										

Project: Project1
Date: Sun 07/06/26 02:58

Task: Inactive Summary: External Tasks:

Split: Manual Task: External Milestone:

Milestone: Duration-only: Deadline:

Summary: Manual Summary Rollup: Progress:

Project Summary: Manual Summary: Manual Progress:

Inactive Task: Start-only:

Inactive Milestone: Finish-only:

Page 2

ID	Task Mode	Task Name	Duration	Start	Finish	28	30	April 2026	01	03	05	07	09	11	13
44		Infrastructure + CI/CD	5 days	Mon 01/06/2	Sun 07/06/2										
45		Pipeline GitHub Actions (6 phases)	2 days	Mon 01/06/2	Tue 02/06/2										
46		OWASP ZAP DAST + Trivy image scan	1 day	Wed 03/06/2	Wed 03/06/2										
47		Build + Push images (GHCR, DockerHub)	1 day	Thu 04/06/2	Thu 04/06/2										
48		Prometheus + Grafana (Monitoring)	2 days	Thu 04/06/2	Sat 06/06/26										
49		Déploiement final + Rapport	1 day	Fri 05/06/26	Sun 07/06/2										

Project: Project1
Date: Sun 07/06/26 02:58

Task: Inactive Summary: External Tasks:

Split: Manual Task: External Milestone:

Milestone: Duration-only: Deadline:

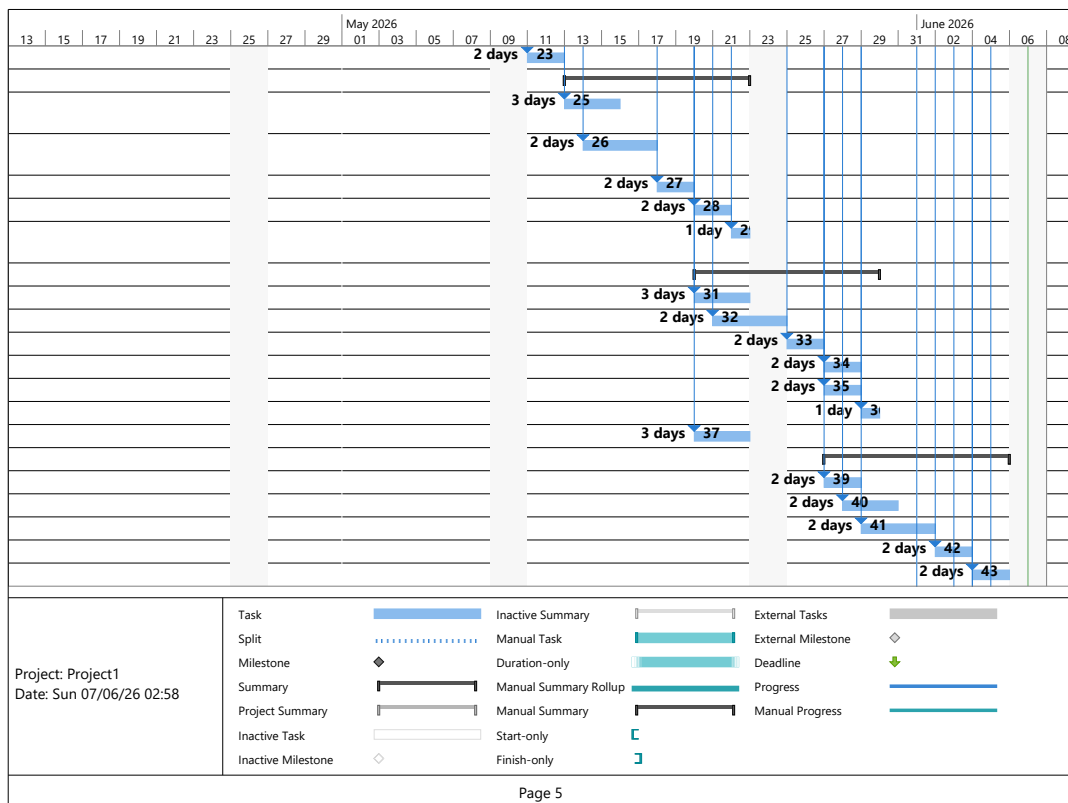
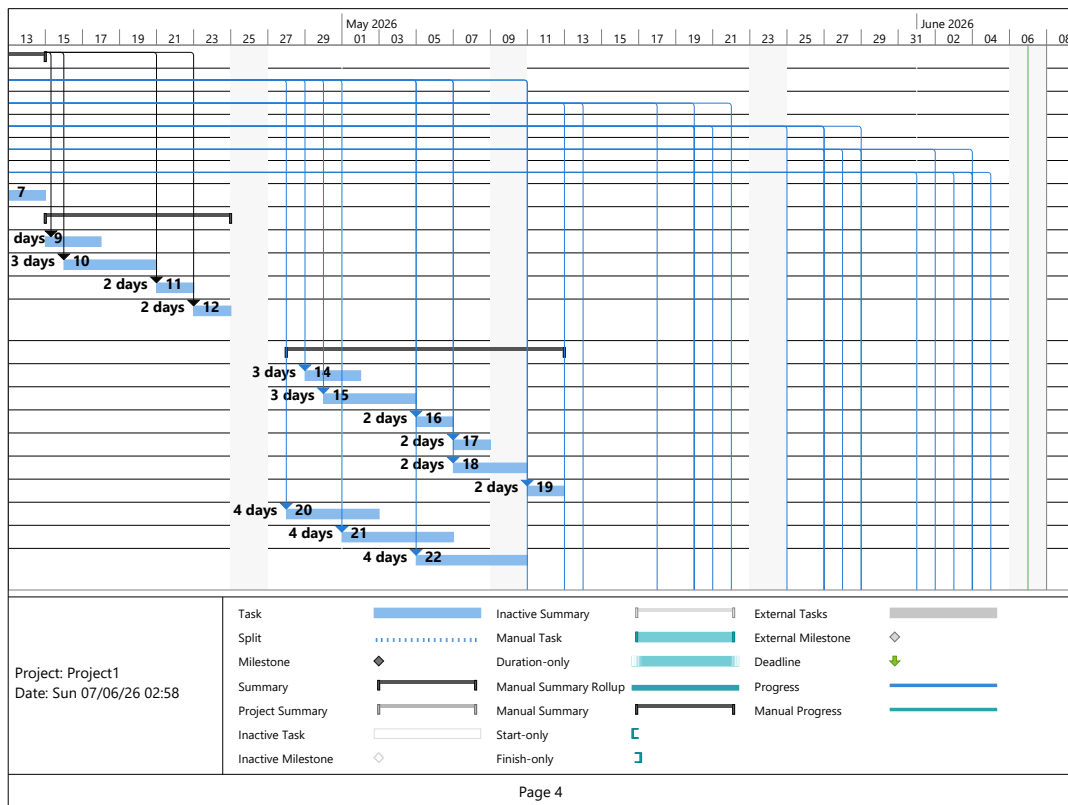
Summary: Manual Summary Rollup: Progress:

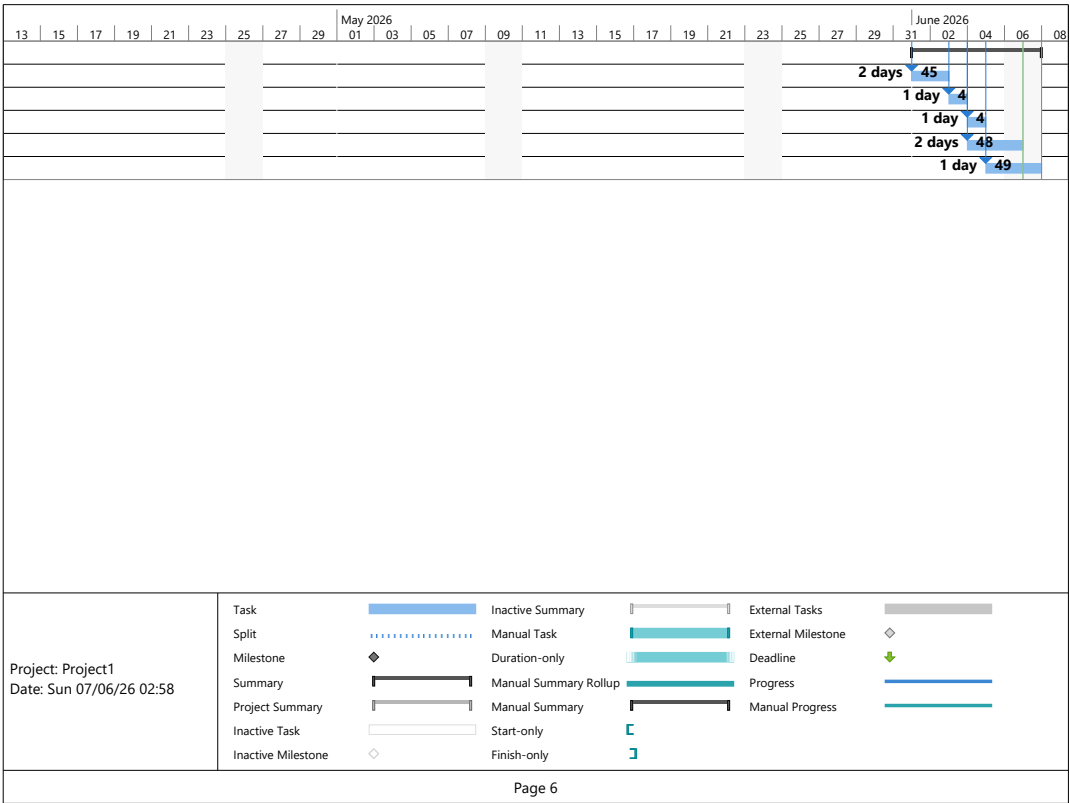
Project Summary: Manual Summary: Manual Progress:

Inactive Task: Start-only:

Inactive Milestone: Finish-only:

Page 3





2.3 Conclusion

Ce premier chapitre a présenté le cadre général du projet **Propel**, les objectifs visés, la méthodologie de gestion adoptée ainsi que l'organisation de l'équipe. Le chapitre suivant sera consacré à l'analyse détaillée des besoins et à la conception du système.

Chapitre 3

Analyse et conception

3.1 Introduction

Ce chapitre présente l’analyse approfondie du système **Propel**. Nous commençons par une étude des solutions existantes, suivie de l’identification des besoins fonctionnels et non fonctionnels. Ensuite, nous identifions les acteurs du système avant de présenter les différents diagrammes de modélisation.

3.2 Étude de l’existant

Plusieurs solutions de création de CV existent sur le marché. Le tableau suivant présente une comparaison des principales solutions :

TABLE 3.1 – Comparaison des solutions existantes

Solution	CV per-sonnalisé	IA	Suivi can-didatures	Open Source
Propel (notre solution)	Oui	Oui	Oui	Oui
Canva	Partiel	Non	Non	Non
Zety	Modèles fixes	Non	Non	Non
Novoresume	Modèles fixes	Non	Non	Non
Rezi	Oui	Oui	Non	Non
Enhancv	Partiel	Oui	Non	Non

Les solutions existantes présentent plusieurs limitations :

- **Personnalisation limitée** : La plupart des outils offrent des modèles fixes sans adaptation réelle au contenu de l’offre d’emploi.

- **Absence de suivi intégré** : Aucune solution ne combine la génération de CV avec le suivi des candidatures dans une plateforme unifiée.
- **Coût** : Les solutions avancées sont payantes (abonnements mensuels).
- **Optimisation ATS limitée** : Peu d'outils optimisent réellement le CV pour passer les filtres des systèmes ATS.

Propel se distingue en offrant une solution complète, gratuite et open source, combinant génération IA de CV, optimisation ATS et suivi de candidatures.

3.3 Étude des besoins fonctionnels

Les besoins fonctionnels identifiés pour le système **Propel** sont les suivants :

TABLE 3.2 – Besoins fonctionnels du système

ID	Description
BF01	L'utilisateur doit pouvoir s'authentifier via Keycloak (email, Google, GitHub)
BF02	L'utilisateur doit pouvoir gérer son portfolio (projets, compétences, expériences, formations)
BF03	L'utilisateur doit pouvoir soumettre une offre d'emploi pour générer un CV personnalisé
BF04	Le système doit extraire les informations clés de l'offre d'emploi via un agent IA
BF05	Le système doit rechercher les compétences pertinentes dans le profil de l'utilisateur via RAG
BF06	Le système doit générer un CV formaté selon un template optimisé ATS
BF07	Le système doit optimiser le contenu du CV pour maximiser la compatibilité ATS
BF08	Le système doit envoyer le CV généré par email à l'utilisateur
BF09	L'utilisateur doit pouvoir suivre ses candidatures (vue Kanban, liste, calendrier)
BF10	L'utilisateur doit pouvoir visualiser des statistiques et des graphiques sur ses candidatures
BF11	Le système doit crawler automatiquement des offres d'emploi depuis des plateformes externes
BF12	L'utilisateur doit recevoir des notifications temps réel sur l'état de ses candidatures
BF13	L'utilisateur doit pouvoir gérer plusieurs versions de CV
BF14	Le système doit exporter les CV au format PDF

3.4 Étude des besoins non fonctionnels

Les besoins non fonctionnels identifiés sont :

- **Performance** : La génération d'un CV ne doit pas excéder quelques secondes. Les temps de réponse de l'API doivent être optimisés.
- **Sécurité** : L'authentification doit être centralisée via Keycloak avec support OAuth2 et OpenID Connect. Les communications doivent être chiffrées (HTTPS). Les vulnérabilités doivent être scannées via Trivy et OWASP ZAP.
- **Disponibilité** : Le système doit être résilient grâce à une architecture en microservices et une orchestration Docker. Les services critiques doivent pouvoir redémarrer automatiquement.
- **Maintenabilité** : Le code source doit être organisé en modules séparés, versionné avec Git, et suivre les principes SOLID. La qualité du code est surveillée via SonarCloud.
- **Évolutivité** : L'architecture microservices permet de faire évoluer chaque service indépendamment. L'utilisation de Kafka permet un couplage faible entre les services.
- **Ergonomie** : L'interface utilisateur doit être intuitive, responsive et suivre une charte graphique cohérente (design system Propel).
- **Portabilité** : L'ensemble de l'application doit être conteneurisée avec Docker pour garantir un déploiement reproductible sur n'importe quel environnement.

3.5 Identification des acteurs

L'analyse du système **Propel** permet d'identifier les acteurs suivants :

TABLE 3.3 – Identification des acteurs du système

Acteur	Description
Utilisateur	Candidat cherchant à générer des CV personnalisés et suivre ses candidatures
Système d'authentification	Keycloak gère l'authentification et l'autorisation via OAuth2/OpenID Connect
Pipeline IA	Ensemble d'agents intelligents qui traitent les offres d'emploi et génèrent les CV
Système de messagerie	Kafka assure la communication asynchrone entre les microservices
Administrateur	Gère la configuration du système et supervise le déploiement

3.6 Modélisation du système

3.6.1 Diagramme de cas d'utilisation

Les diagrammes de cas d'utilisation ci-dessous présentent les interactions entre l'utilisateur et les différentes fonctionnalités du système. Cinq diagrammes couvrent les principaux modules de l'application.

Authentification L'utilisateur peut créer un compte, se connecter et se déconnecter du système.

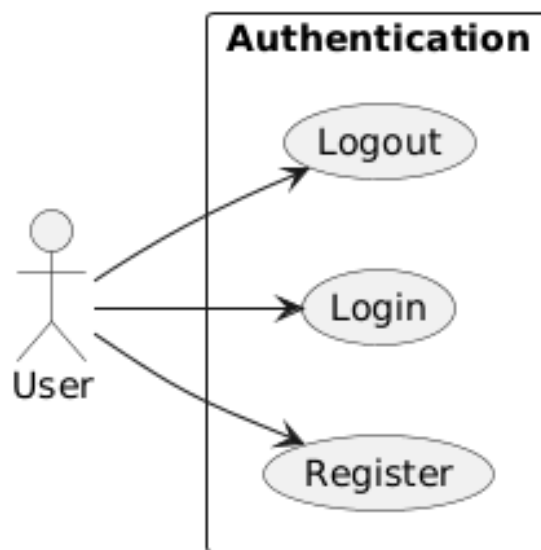


FIGURE 3.1 – Cas d'utilisation — Authentification

Gestion du profil L'utilisateur peut gérer l'ensemble de ses données personnelles et professionnelles : informations de profil, projets, compétences, expériences, formations, certifications, langues, liens sociaux et profils CV. Tous ces cas nécessitent une authentification préalable.

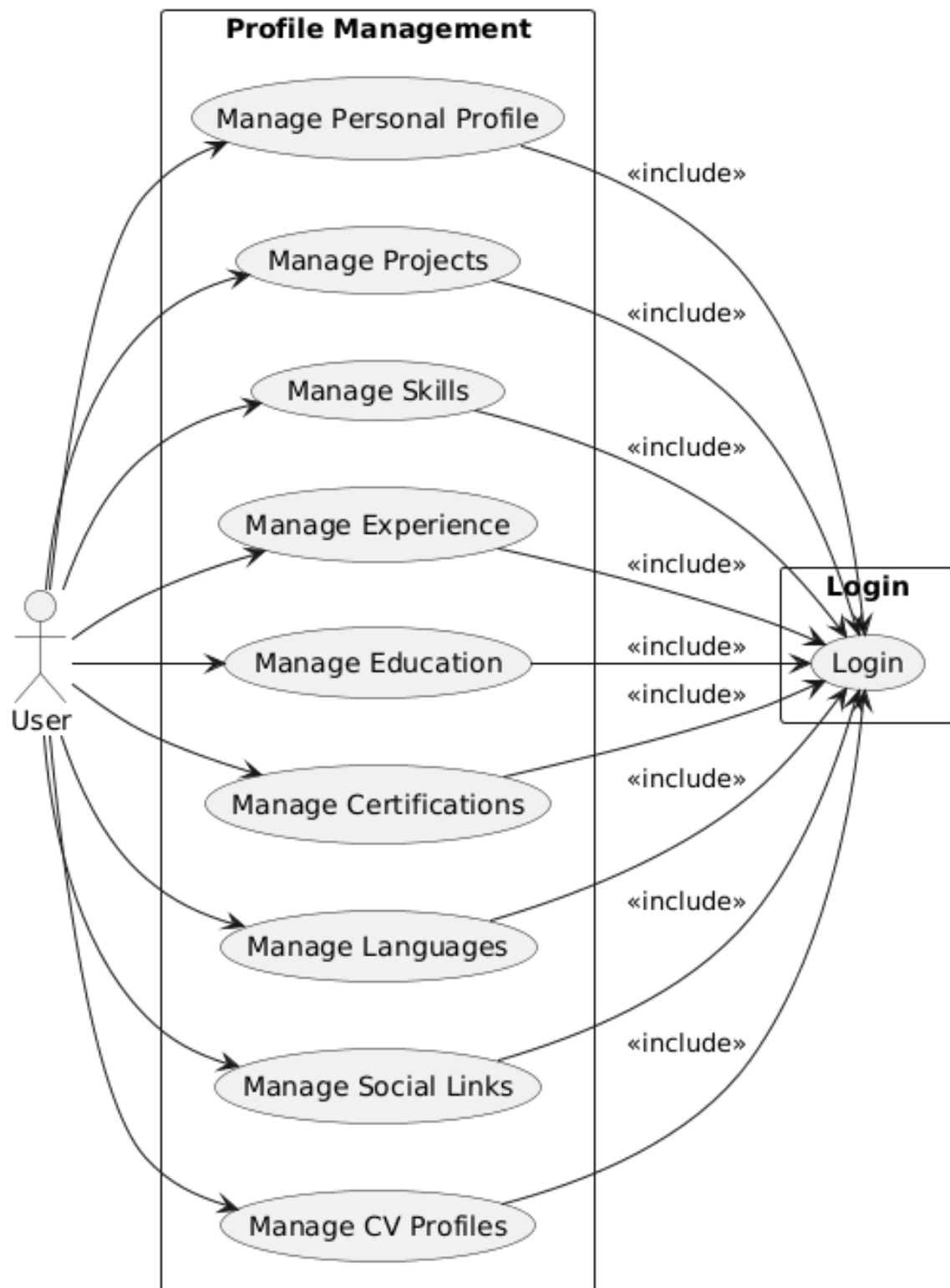


FIGURE 3.2 – Cas d'utilisation — Gestion du profil

Génération de CV L'utilisateur peut générer un CV personnalisé à partir d'une offre d'emploi, choisir un template, définir le ton du contenu, exporter le CV au format PDF et consulter l'historique des générations. Le système d'IA intervient comme acteur

secondaire dans le processus de génération.

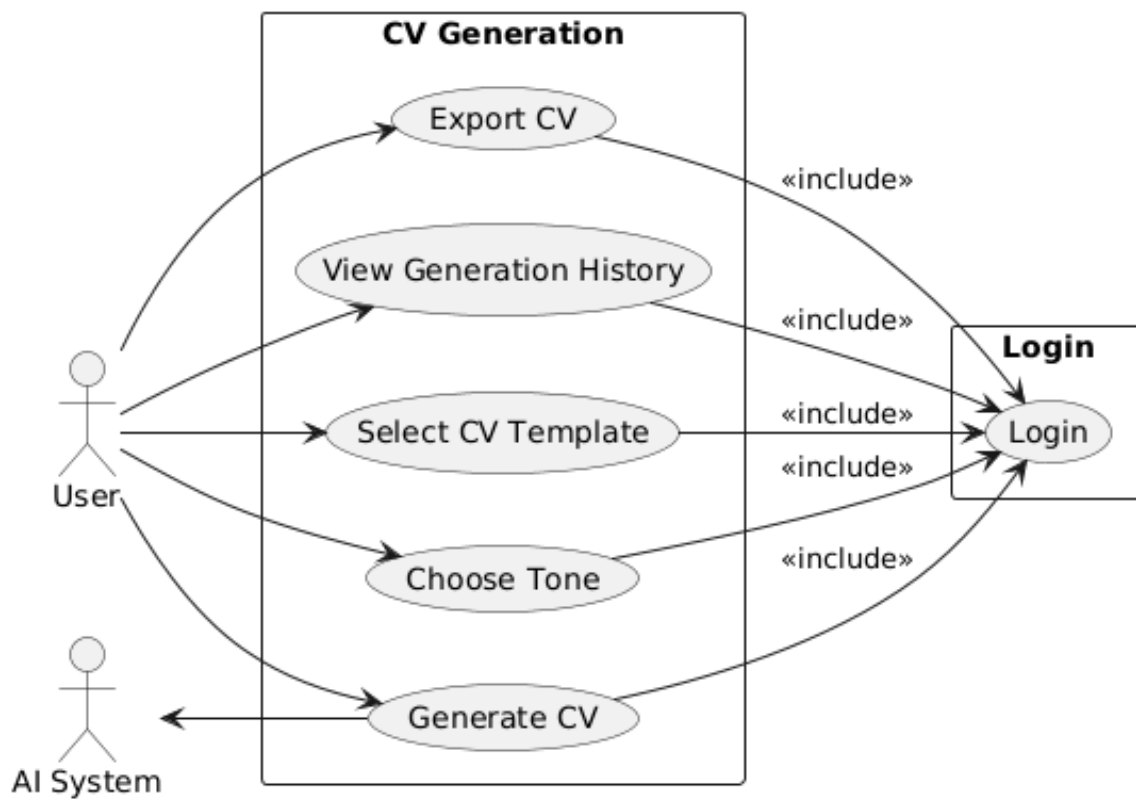


FIGURE 3.3 – Cas d’utilisation — Génération de CV

Suivi des candidatures L'utilisateur peut ajouter une candidature, mettre à jour son statut, visualiser le tableau Kanban, consulter les analyses statistiques et le calendrier des événements.

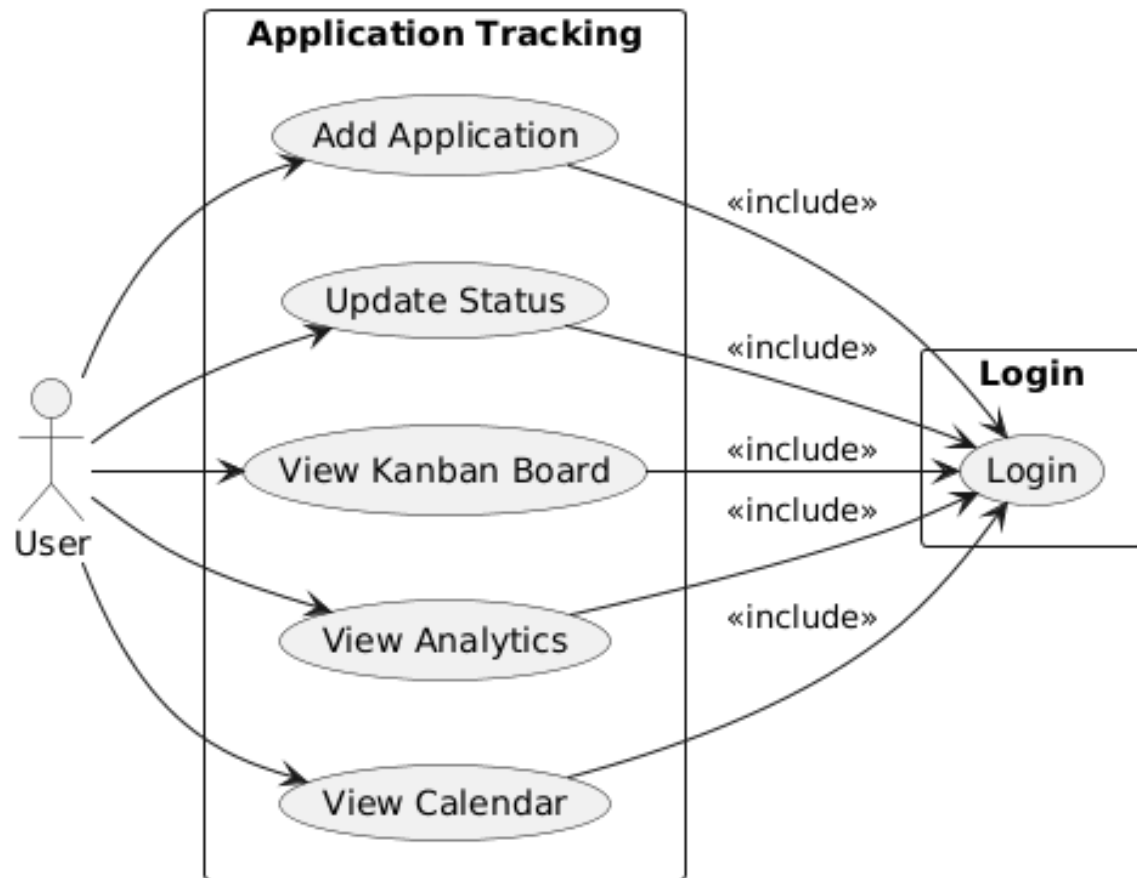


FIGURE 3.4 – Cas d'utilisation — Suivi des candidatures

Offres d'emploi et notifications L'utilisateur peut parcourir les offres d'emploi, déclencher un crawling en direct, configurer ses notifications, planifier des rappels et programmer des envois d'emails.

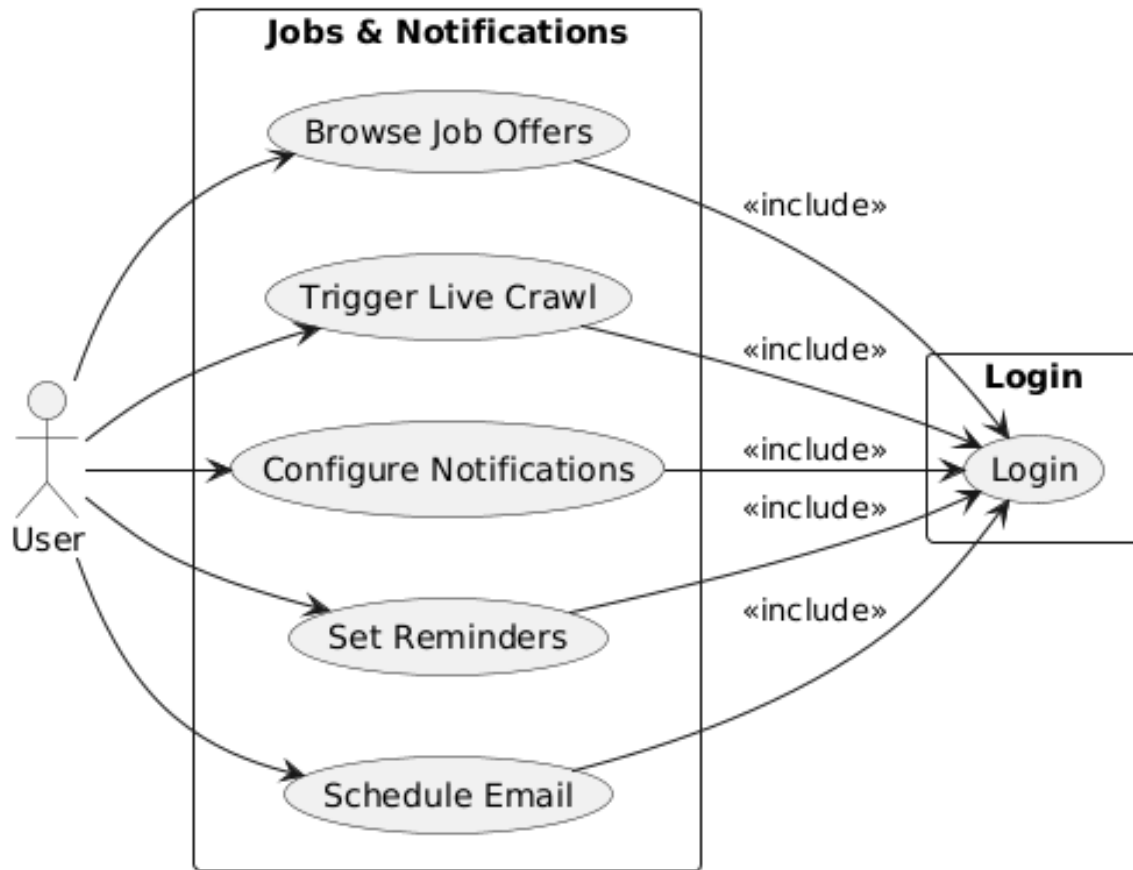


FIGURE 3.5 – Cas d’utilisation — Offres d’emploi et notifications

3.6.2 Diagramme de classes

Le diagramme de classes ci-dessous illustre les relations entre les entités principales du système, organisées par base de données. Chaque package coloré représente un microservice et sa base de données associée. Les relations montrent les dépendances inter-services, comme les références de l’utilisateur vers les contenus, candidatures et CV, ou les liens entre une candidature et une version de CV.

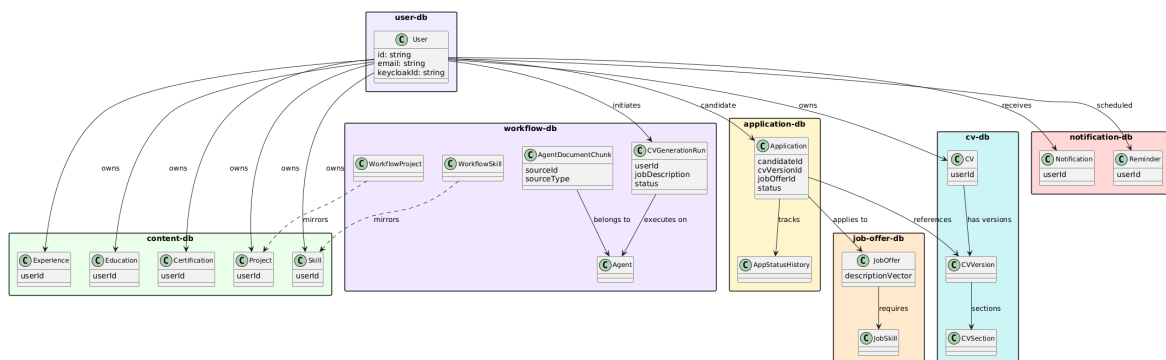


FIGURE 3.6 – Diagramme de classes — Relations inter-services

3.6.3 Diagramme de séquence — Flux d'authentification

Le diagramme de séquence ci-dessous illustre le flux d'authentification via Keycloak avec le protocole OpenID Connect (OIDC). L'utilisateur est redirigé vers Keycloak pour s'authentifier, reçoit un code d'autorisation, puis l'API Gateway échange ce code contre des jetons JWT. Un cookie de session (`cv_session`) est créé pour maintenir la session. Lors de l'inscription, un événement `user.registered` est publié sur Kafka pour notifier les autres services.

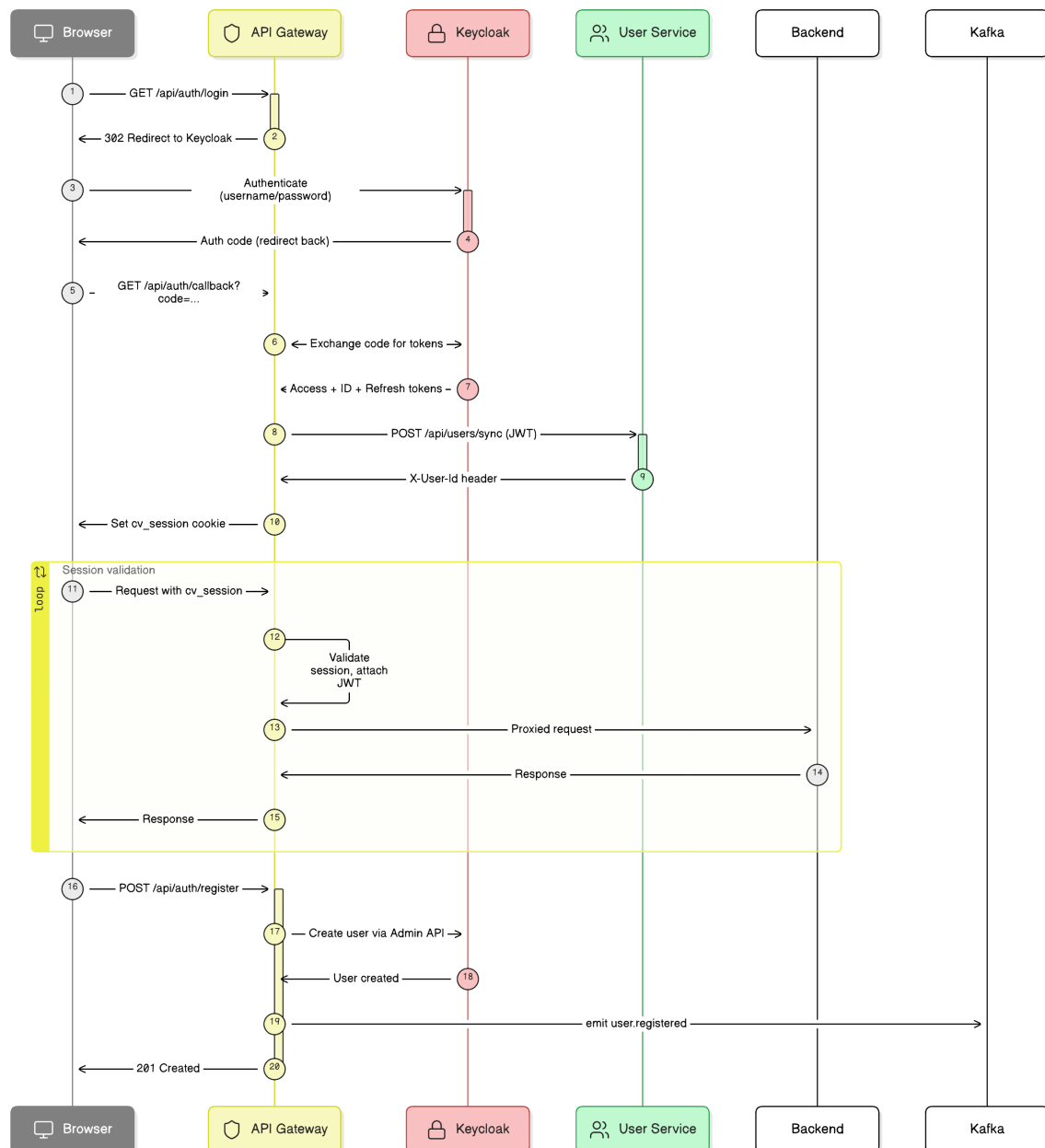


FIGURE 3.7 – Diagramme de séquence — Flux d'authentification OIDC

3.6.4 Modèle relationnel de la base de données

L'architecture microservices du projet implique l'utilisation de plusieurs bases de données PostgreSQL 16, une par service. Chaque base de données est spécialisée pour les besoins de son service et réside dans un conteneur Docker dédié. L'extension pg-vector est utilisée pour le support des recherches vectorielles dans les bases concernées par le pipeline IA.

Base Utilisateurs (user-db — port 5433)

La base utilisateurs centralise les informations personnelles et professionnelles des candidats.

- **users** : Table principale contenant le profil complet de chaque utilisateur. Les attributs incluent `keycloakId` (identifiant de synchronisation avec Keycloak), `headline` (titre professionnel), `desiredSalaryMin/desiredSalaryMax` (prétentions salariales), `requiresVisaSponsorship`, `noticePeriod` (préavis), `employmentTypes` (types d'emploi souhaités), `remotePreference` (préférence de télétravail), `willingToRelocate` (mobilité géographique). Des colonnes JSON permettent de stocker des données flexibles : `preferencesJson` et `aiProfileDataJson` (profil généré par l'IA).

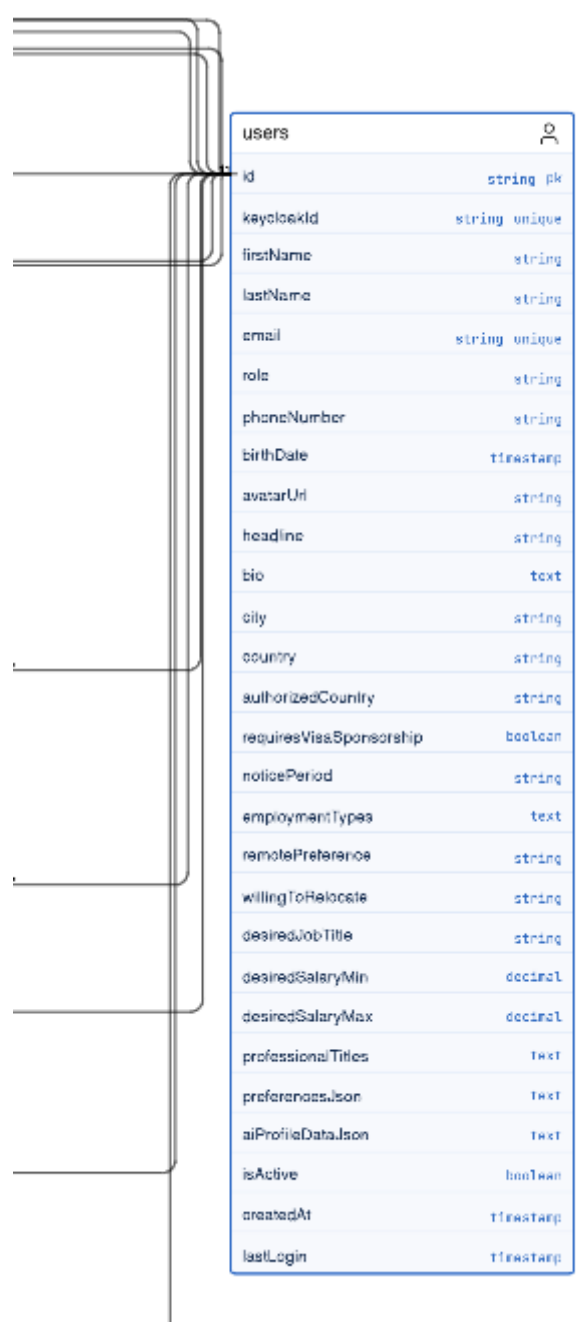


FIGURE 3.8 – Modèle relationnel — Base Utilisateurs

Base Contenu Utilisateur (content-db — port 5434)

Cette base gère l'ensemble du portfolio de l'utilisateur. Chaque table est reliée à l'utilisateur via une clé étrangère `userId`.

- **projects** : Projets réalisés. L'attribut `skillsJson` stocke les compétences associées sous forme JSON. `repositoryUrl` et `demoUrl` pointent vers les sources et démonstrations.
- **skills** : Compétences avec `category` (langage, framework, outil), `level` (débutant, intermédiaire, avancé, expert) et `yearsOfExperience`.

- **experiences** : Expériences professionnelles avec **company** et **status** (en cours, terminée).
- **educations** : Formations avec **degreeType** (licence, master, doctorat), **fieldOfStudy** et **specialization**. **diplomaFileUrl** stocke le lien vers le fichier du diplôme.
- **certifications** : Certifications avec **issuingOrganization** et **credentialUrl**.
- **languages** : Langues avec **level** (A1 à C2).
- **interests** : Centres d'intérêt.
- **hackathons** : Participations à des hackathons avec **role**, **result** et **description**.
- **socialLinks** : Liens vers les réseaux sociaux (**platform**, **url**).
- **cvProfiles** : Profils de CV avec **summary** (résumé personnel) et **title**.
- **academicActivities** : Activités académiques (projets scolaires, publications) avec période.



FIGURE 3.9 – Modèle relationnel — Base Contenu Utilisateur

Base Candidatures (application-db — port 5436)

Cette base assure le suivi des candidatures des utilisateurs.

- **applications** : Candidatures avec **candidateId**, **cvVersionId** et **jobOfferId** comme clés étrangères. **companyName** et **positionTitle** décrivent le poste visé.

`status` peut prendre les valeurs : `saved`, `applied`, `interview`, `offer`, `rejected`. `offerSource` indique la provenance de l'offre. `isSaved` permet de marquer une candidature comme favorite.

- `applicationStatusHistory` : Historique des changements de statut. `oldStatus` et `newStatus` tracent chaque transition. `changedBy` enregistre l'acteur (utilisateur ou système) et `comment` permet d'ajouter une note.
- `applicationConfigurations` : Préférences utilisateur pour le module. `showReminders` active les rappels, `selectedStatuses` filtre les statuts affichés.

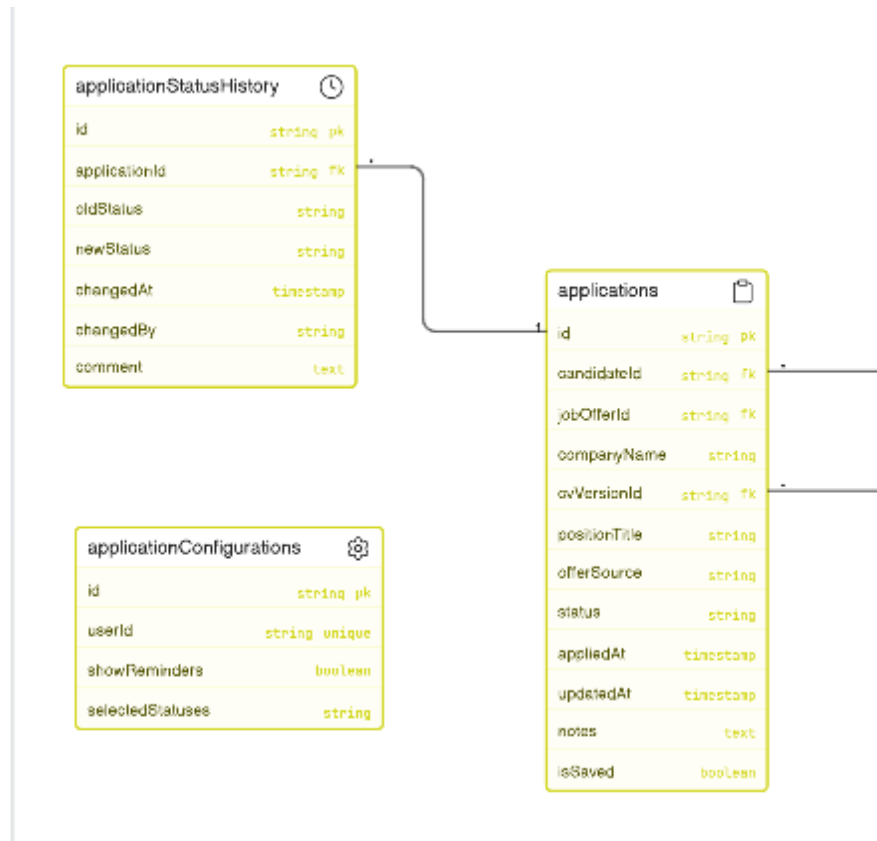


FIGURE 3.10 – Modèle relationnel — Base Candidatures

Base Workflow (workflow-db — port 5435)

La base workflow est la plus complexe. Elle pilote le pipeline de génération de CV et utilise pgvector pour les recherches vectorielles.

- `workflows` : Définition des pipelines. `definitionJson` contient la configuration du workflow au format JSON. `isActive` active ou désactive le pipeline.
- `cvGenerationRuns` : Traces d'exécution de chaque génération de CV. `jobDescription` stocke le texte brut de l'offre. `status` suit l'état (`pending`, `running`, `completed`, `failed`, `cancelled`). `stepStatuses` est un champ JSON contenant l'état de chaque étape. Les résultats des agents sont stockés dans des colonnes dédiées : `extractionResult`, `searchResult`, `renderResult`, `optimizationResult`, `deliveryResult`.

- **agents** : Configuration des agents IA. **agentId** est un identifiant unique (ex : job-extractor). **sortOrder** détermine l'ordre d'exécution dans le pipeline. **backgroundGradient** est utilisé pour l'affichage dans l'interface.
- **agentDocumentChunks** : Stockage des documents vectorisés pour le RAG. **sourceId** et **sourceType** identifient l'entité source. **embedding** est un vecteur pgvector utilisé pour la recherche sémantique. **searchVector** est un tsvector pour la recherche full-text.
- **jobExtractions** : Extractions d'offres d'emploi. **inputType** distingue le texte saisi de l'URL. **confidence** est un score de confiance de l'extraction. **outputJson** contient le résultat structuré.
- **workflowProjects**, **workflowSkills**, **workflowExperiences** : Entités miroir du contenu utilisateur, optimisées pour la recherche vectorielle avec des champs **embedding** et **aiSummaryJson**.



FIGURE 3.11 – Modèle relationnel — Base Workflow

Base Offres d'Emploi (job-offer-db — port 5437)

Cette base stocke les offres d'emploi collectées par l'agent de crawling et enrichies par l'agent d'extraction.

- **jobOffers** : Offres d'emploi. **jobHash** est un hash unique de l'offre pour éviter les doublons lors du crawling. **sourceUrl** est l'URL d'origine. **rawDescription** stocke la description brute. **descriptionVector** est un vecteur pgvector pour le matching avec le profil utilisateur. **locationType** distingue **remote**, **onsite** et **hybrid**. **employmentType** (**full-time**, **part-time**, **contract**) et **seniorityLevel**

- (junior, mid, senior, lead) qualifie le poste.
- **jobSkills** : Compétences requises pour le poste. **type** distingue les compétences **required** et **preferred**. **isMandatory** indique si la compétence est obligatoire.
- **jobResponsibilities** : Liste des responsabilités du poste.
- **jobBenefits** : Avantages proposés par l'entreprise.
- **searchCaches** : Cache des recherches de crawling. **keyword** et **location** définissent la recherche. **status** suit l'avancement. **expectedCount** et **processedCount** permettent de suivre le nombre d'offres traitées.
- **userQuotas** : Quotas de crawling par utilisateur. **lastCrawlDate** enregistre la dernière date d'utilisation.

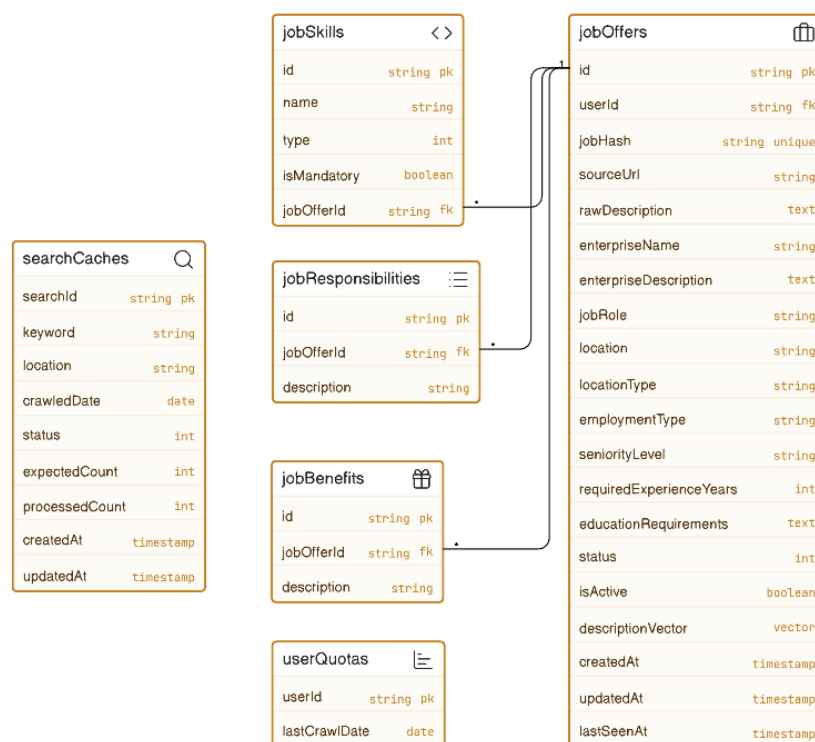


FIGURE 3.12 – Modèle relationnel — Base Offres d'Emploi

Base CV (cv-db — port 5439)

La base CV gère les curriculum vitae et leurs versions successives.

- **cvs** : CV avec **templateId** (identifiant du template utilisé) et **isActive** (CV actif par défaut).
- **cvVersions** : Versions d'un CV. **versionNumber** est un numéro incrémental. **contentJson** stocke le contenu complet du CV au format JSON structuré. **fileUrl** et **pdfUrl** pointent respectivement vers le fichier source et le PDF généré stockés dans MinIO.
- **cvSections** : Sections d'une version de CV. **sectionType** définit le type (**education**,

experience, skills, projects, certifications, languages). `contentJson` contient les données de la section. `displayOrder` détermine l'ordre d'affichage.

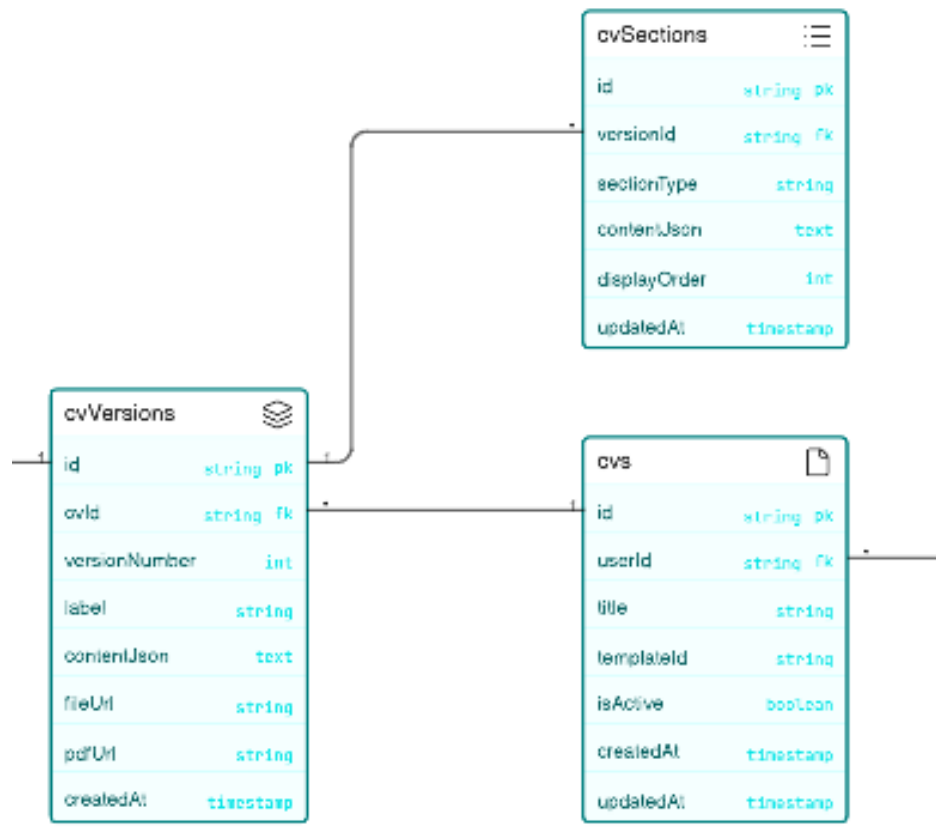


FIGURE 3.13 – Modèle relationnel — Base CV

Base Notifications (notification-db — port 5438)

Cette base gère l'ensemble du système de notifications, de rappels et de messagerie.

- **notifications** : Notifications in-app et email. `type` définit la catégorie (`application_update`, `cv_ready`, `reminder`, `welcome`). `channel` détermine le mode (`email`, `in-app`). `isRead` et `isSent` tracent l'état. `errorMessage` enregistre les échecs d'envoi.
- **notificationPreferences** : Préférences de notification par utilisateur. `enableEmail` et `enableInApp` activent les canaux. `applicationUpdates`, `cvUpdates`, `reminders`, `weeklyDigest` activent les catégories. `defaultReminderDaysBefore` définit le délai de rappel par défaut.
- **reminders** : Rappels planifiés. `eventDate` est la date de l'événement, `reminderAt` est la date du rappel. `reminderOffset` définit le décalage (ex : 1d, 2h). `status` suit l'état (`pending`, `sent`, `cancelled`).
- **contacts** : Carnet d'adresses. `source` indique la provenance (`manuel`, `import Gmail`). `isFavorite` permet de marquer les contacts favoris. `avatarBase64`

stocke l'avatar encodé.

- **emailMessages** : Historique des emails envoyés. **provider** indique le fournisseur (MailKit, Gmail API). **status** suit la délivrabilité. **contactId** relie l'email à un contact. **scheduleId** permet de lier à une campagne planifiée.
- **emailSchedules** : Campagnes d'emails planifiées. **cronExpression** définit la récurrence. **recipientIds** stocke les destinataires. **isActive** active ou désactive la campagne.
- **gmailConnections** : Connexions Gmail des utilisateurs. **encryptedAccessToken** et **encryptedRefreshToken** stockent les jetons OAuth chiffrés. **isRevoked** indique si la connexion a été révoquée.



FIGURE 3.14 – Modèle relationnel — Base Notifications

3.7 Conclusion

Ce chapitre a présenté l'analyse complète du système **Propel**, incluant l'étude de l'existant, les besoins fonctionnels et non fonctionnels, l'identification des acteurs et la modélisation. Le chapitre suivant détaillera la réalisation technique du projet.

Chapitre 4

Réalisation

4.1 Introduction

Ce chapitre présente la réalisation technique du projet **Propel**. Nous détaillons l'architecture physique et logicielle, les outils utilisés, le cycle DevSecOps mis en place, la gestion du code source, l'implémentation des différentes couches de l'application, ainsi que le déploiement.

4.2 Étude technique

4.2.1 Architecture physique

Infrastructure matérielle

Le projet s'appuie sur une infrastructure locale déployée sur des machines virtuelles VMware Workstation, provisionnées via Terraform. Chaque membre de l'équipe est responsable de VMs dédiées selon la répartition des services.

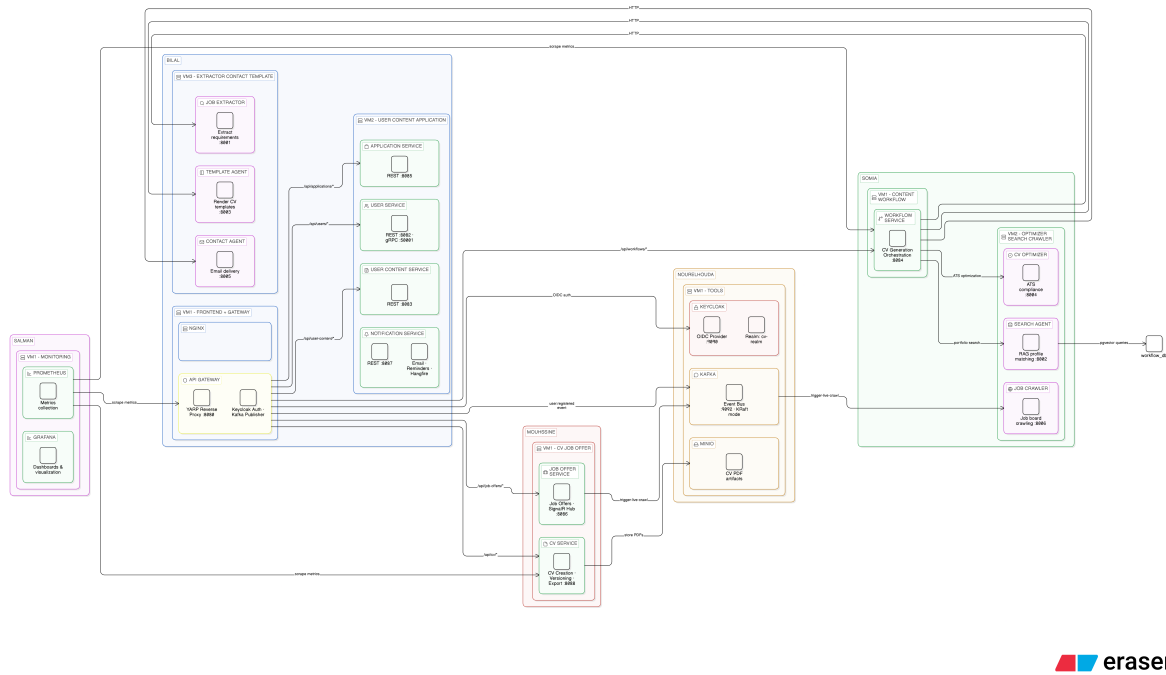


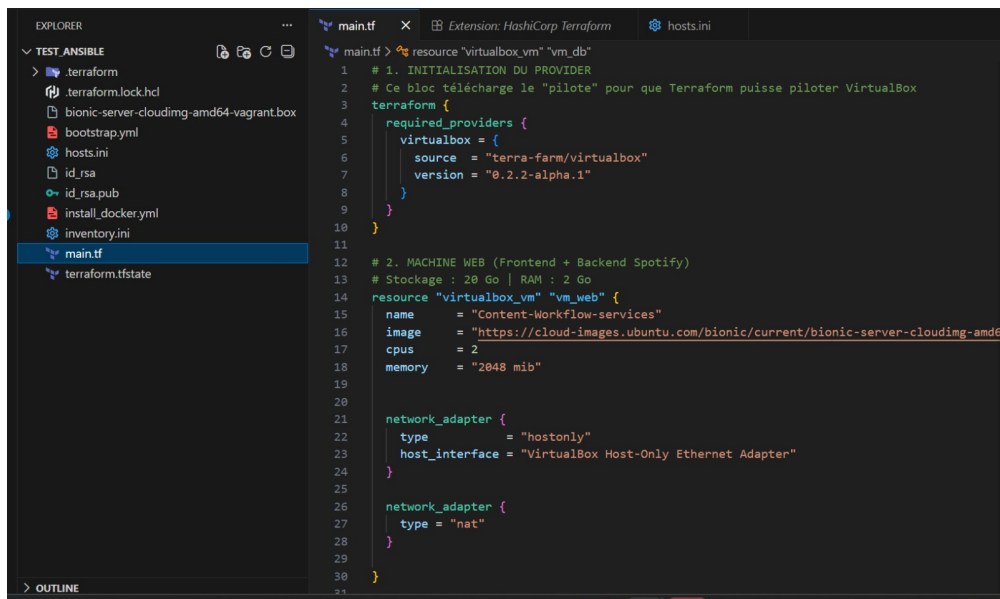
FIGURE 4.1 – Architecture de déploiement locale — Répartition des VMs par membre

La répartition des VMs est la suivante :

- **Bilal (3 VMs)** : Frontend + Gateway, Services applicatifs (User, Content, Application, Notification), Agents IA (Extractor, Contact, Template).
- **Somia (2 VMs)** : Service Workflow avec base pgvector, Agents IA (Optimizer, Search, Crawler).
- **Norelhouda (1 VM)** : Infrastructure transverse (Keycloak, Kafka, MinIO).
- **Mouhssine (1 VM)** : Service CV et Service Offres d'Emploi.
- **Salman (1 VM)** : Monitoring (Prometheus, Grafana).

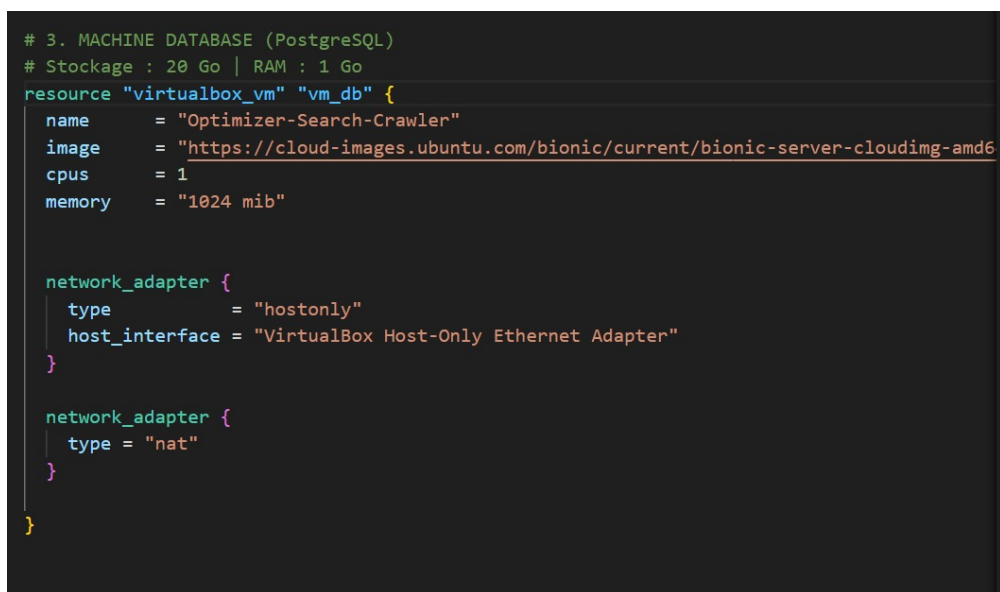
Chaque VM dispose de 2 CPU et 2 Go de RAM, et les services communiquent entre VMs via le réseau partagé Docker.

Le provisionnement des VMs est automatisé à l'aide de **Terraform** avec le provider VMware Workstation. Les scripts Terraform définissent la configuration de chaque VM (CPU, RAM, stockage) et sont versionnés dans le dépôt du projet.



```
1 resource "virtualbox_vm" "vm_db"
2 # 1. INITIALISATION DU PROVIDER
3 # Ce bloc télécharge le "pilote" pour que Terraform puisse piloter VirtualBox
4 terraform {
5   required_providers {
6     virtualbox = {
7       source = "terra-farm/virtualbox"
8       version = "0.2.2-alpha.1"
9     }
10  }
11 }
12 # 2. MACHINE WEB (Frontend + Backend Spotify)
13 # Stockage : 20 Go | RAM : 2 Go
14 resource "virtualbox_vm" "vm_web" {
15   name = "Content-Workflow-services"
16   image = "https://cloud-images.ubuntu.com/bionic/current/bionic-server-cloudimg-amd64"
17   cpus = 2
18   memory = "2048 mib"
19
20   network_adapter {
21     type = "hostonly"
22     host_interface = "VirtualBox Host-Only Ethernet Adapter"
23   }
24
25   network_adapter {
26     type = "nat"
27   }
28 }
29
30 }
```

FIGURE 4.2 – Script Terraform — Définition des ressources VMs



```
# 3. MACHINE DATABASE (PostgreSQL)
# Stockage : 20 Go | RAM : 1 Go
resource "virtualbox_vm" "vm_db" {
  name = "Optimizer-Search-Crawler"
  image = "https://cloud-images.ubuntu.com/bionic/current/bionic-server-cloudimg-amd64"
  cpus = 1
  memory = "1024 mib"

  network_adapter {
    type = "hostonly"
    host_interface = "VirtualBox Host-Only Ethernet Adapter"
  }

  network_adapter {
    type = "nat"
  }
}
```

FIGURE 4.3 – Script Terraform — Configuration réseau et déploiement

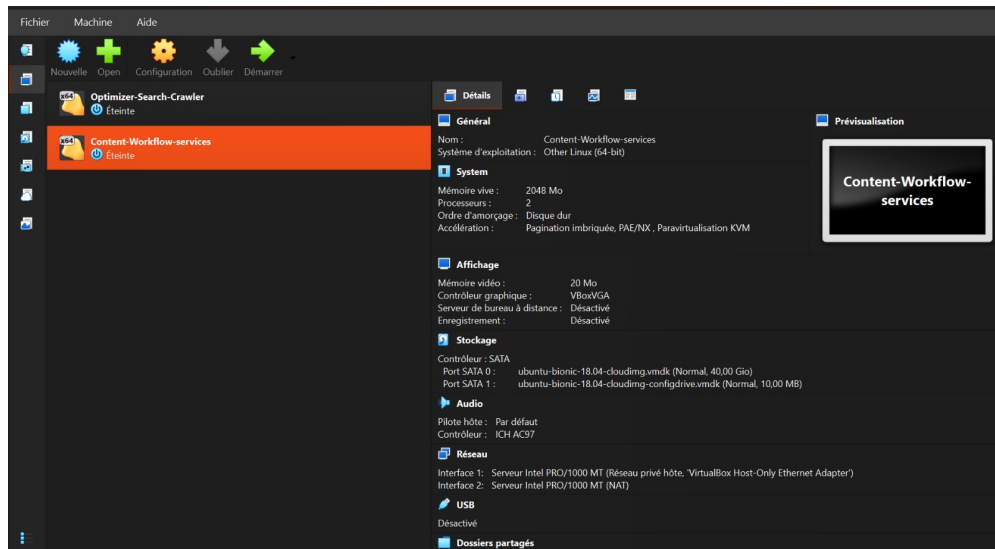


FIGURE 4.4 – Résultat du provisionnement — VMs créées

Automatisation avec Ansible

La configuration des VMs après provisionnement est automatisée via **Ansible**. Les playbooks Ansible installent Docker, Docker Compose, pullent les images des services et assurent la configuration réseau de chaque VM.

```
- name: Installation de Docker en mode RAW (Sans aucun Python)
  hosts: monitoring
  become: yes
  gather_facts: no

  tasks:
    - name: Installation de Docker via commande SSH brute
      raw: |
        export DEBIAN_FRONTEND=noninteractive
        apt-get update && \
        apt-get install -y apt-transport-https ca-certificates curl software-properties-c
        curl -fsSL https://download.docker.com/linux/ubuntu/gpg | apt-key add - && \
        add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu bio
        apt-get update && \
        apt-get install -y docker-ce docker-ce-cli containerd.io
      register: raw_output

    - name: Verifier si Docker est installékj
      raw: docker --version
      register: docker_check

    - name: Affichage du résultat
      debug:
        msg: "Succès ! {{ docker_check.stdout }}"
```

FIGURE 4.5 – Playbook Ansible pour la configuration des VMs


```

changed: [192.168.56.110]
changed: [192.168.56.109]
changed: [192.168.56.108]

TASK [Start Docker service] *****
*****
ok: [192.168.56.109]
ok: [192.168.56.110]
ok: [192.168.56.108]

TASK [Add user to docker group] *****
*****
changed: [192.168.56.109]
changed: [192.168.56.110]
changed: [192.168.56.108]

TASK [Install Docker Compose] *****
*****
changed: [192.168.56.108]
changed: [192.168.56.110]
changed: [192.168.56.109]

PLAY RECAP *****
*****
192.168.56.108      : ok=10    changed=8    unreachable=0
    failed=0    skipped=0    rescued=0    ignored=0
192.168.56.109      : ok=10    changed=8    unreachable=0
    failed=0    skipped=0    rescued=0    ignored=0
192.168.56.110      : ok=10    changed=8    unreachable=0
    failed=0    skipped=0    rescued=0    ignored=0

```

FIGURE 4.6 – Logs d'exécution d'Ansible

Infrastructure cloud

Parallèlement à l'infrastructure locale, le projet **Propel** a été conçu pour être déployé sur le cloud **AWS** via une approche [Infrastructure as Code \(IaC\)](#) complète. L'infrastructure est intégralement définie dans le répertoire `infrastructure/cloud/` et se compose de deux couches : le provisionnement avec **Terraform** et le déploiement applicatif avec **Ansible**.

Architecture réseau. Un [Virtual Private Cloud \(VPC\)](#) AWS héberge l'ensemble des ressources, avec deux sous-réseaux publics (ALB, NAT Gateway) et deux sous-réseaux privés (instances ECS, RDS). Une passerelle NAT unique assure la connectivité sortante des instances privées vers Internet (téléchargement d'images ECR, appels aux API LLM, envoi SMTP). Un [Application Load Balancer \(ALB\)](#) écoute sur le port 80 et achemine le trafic selon les règles suivantes :

- `/api/*` → service `api-gateway` (port 8080) ;
- `/realms/*`, `/resources/*`, `/admin/*` → service `keycloak` (port 8080) ;
- tout autre chemin → service `frontend` (port 80).

Orchestration ECS. Les services sont exécutés sur un cluster **Amazon ECS** avec le lancement **EC2** (type `m7i-flex.large`, 8 GiB), et non Fargate, conformément au cahier des charges. Un **Auto Scaling Group (ASG)** couplé à un **capacity provider** gère le dimensionnement. Le mode réseau **awsvpc** est utilisé avec le **ENI trunking** activé, permettant à chaque instance d’accueillir une dizaine de tâches. La résolution de noms inter-services est assurée par **ECS Service Connect** via un espace de noms DNS privé (`cvgen.local`), remplaçant la résolution Docker Compose de l’environnement local.

Registre d’images. Chacun des 15 services applicatifs (frontend, passerelle API, 7 microservices .NET, 6 agents IA) ainsi que Keycloak possède son propre dépôt **ECR**. Les images sont construites et poussées par Ansible via la tâche `images`.

Base de données. Une instance unique **RDS PostgreSQL 16** (`db.t3.micro`) héberge l’ensemble des 8 bases de données (7 bases applicatives + `keycloak`). L’extension **pgvector** est activée sur les bases `workflow_db` et `cv_db` pour le **Retrieval-Augmented Generation (RAG)**. La base est déployée en simple zone de disponibilité (single-AZ), sans accès public. Les 8 bases sont créées par Ansible via un tunnel SSM, car RDS ne crée qu’une seule base initiale au provisionnement.

Gestion des secrets. Les informations sensibles (mots de passe, clés API, chaînes de connexion) sont stockées dans **AWS SSM Parameter Store** et injectées dans les définitions de tâches ECS au lancement. Les variables dépendant du DNS public de l’ALB (adresses externes des services) sont d’abord créées par Terraform comme paramètres fictifs, puis écrasées par Ansible une fois le DNS connu.

Services éphémères. Par choix d’architecture (démon), **Kafka** (mode KRaft) et **MinIO** sont déployés avec un stockage éphémère — un redémarrage entraîne la perte des événements en transit et des CVs stockés, qui peuvent être régénérés facilement.

Automatisation Ansible. Le playbook Ansible (`playbook.yml`) orchestre le déploiement en plusieurs étapes :

1. **images** : construction et push des images Docker vers ECR ;
2. **secrets** : écriture des paramètres SSM et mise à jour du DNS de l’ALB ;
3. **database** : création des 8 bases de données et activation de `pgvector` (via tunnel SSM vers RDS) ;
4. **deploy** : déploiement des définitions de tâches et services ECS ;
5. **smoketest** : validation du bon fonctionnement de chaque service.

Taille *free-plan*. L'infrastructure est dimensionnée pour fonctionner dans les limites du programme **AWS Free Plan** (crédits 2025 jusqu'à 200 \$) : les instances EC2 (`m7i-flex.large`) et RDS (`db.t3.micro`) sont éligibles, de même que l'ALB. Les coûts récurrents les plus significatifs sont la passerelle NAT (environ 32 \$/mois) et les instances EC2, qui peuvent être arrêtées hors démonstration.

Environnement de déploiement

L'ensemble de l'application est conteneurisé à l'aide de **Docker**. Chaque service possède son propre Dockerfile avec une construction multi-stage optimisée. L'orchestration est assurée par **Docker Compose** au niveau racine du projet, qui inclut trois fichiers compose spécialisés :

- **backend/docker-compose.yml** : Contient les 7 microservices backend .NET, les 8 bases de données PostgreSQL, Kafka, et MinIO.
- **ai_agents/docker-compose.yml** : Contient le pipeline de 6 agents IA en Python avec leur orchestrateur.
- **api_gateway/docker-compose.yml** : Contient la passerelle API (YARP) et Keycloak.

Tous les services partagent un réseau Docker commun (`cv-network`) pour la communication inter-services.

4.2.2 Architecture logicielle

L'architecture logicielle de **Propel** suit un modèle en **microservices** avec communication événementielle. Le schéma ci-dessous illustre l'architecture globale du système.

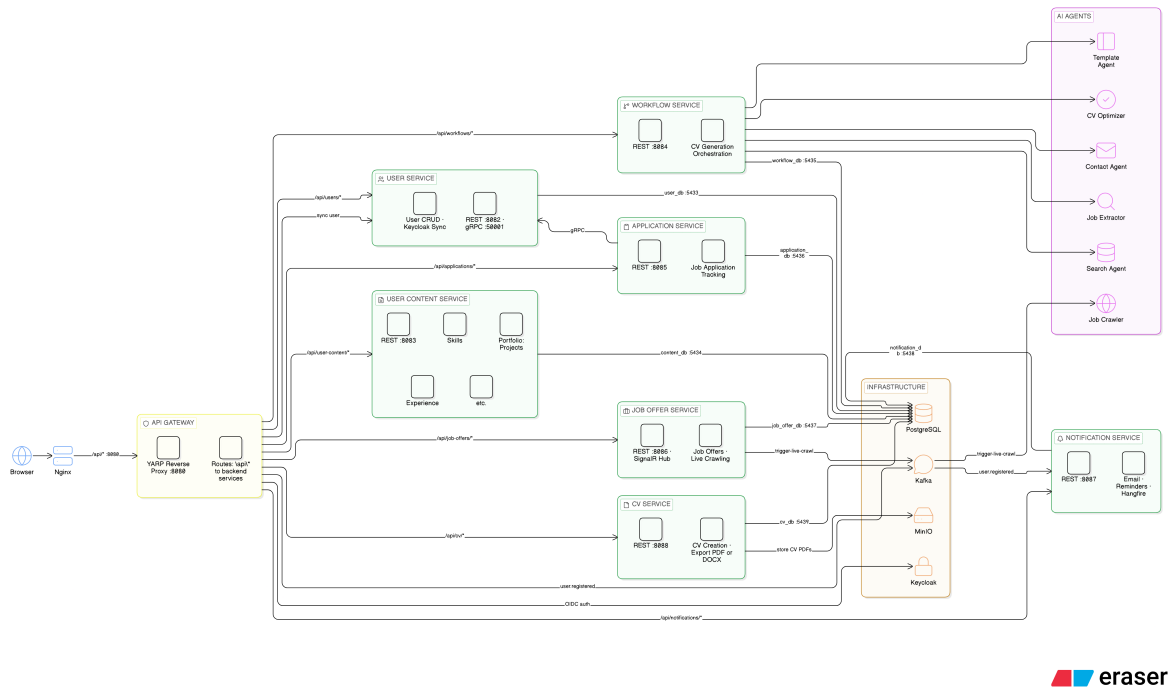


FIGURE 4.7 – Architecture logicielle du système

Les sections suivantes détaillent chaque couche de l'architecture.

Architecture du Frontend

Le frontend est une application monopage ([Single Page Application \(SPA\)](#)) développée avec **Angular 21** en utilisant les **standalone components**. La gestion asynchrone est assurée par **RxJS** et la réactivité par les **Signals** d'Angular. L'interface utilise le design system **Propel** avec des couleurs en espace OKLCH. L'application est hébergée par un serveur **Nginx** servant les fichiers compilés.

Le code source est organisé selon les bonnes pratiques Angular avec une séparation claire entre les **components**, **services**, **pipes** et **guards**. La structure des dossiers est présentée dans la capture ci-dessous.

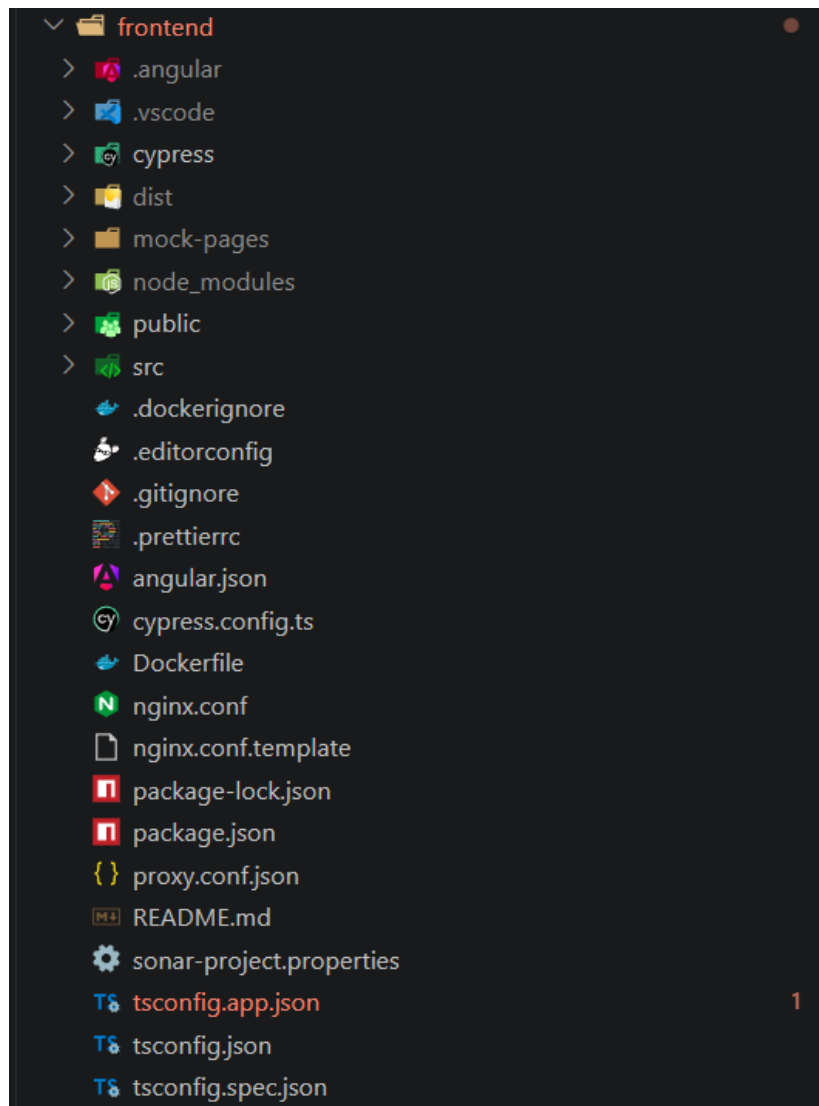


FIGURE 4.8 – Structure du projet Frontend Angular

Passerelle API (YARP)

La passerelle API est un service **ASP.NET Core 10** utilisant [Yet Another Reverse Proxy \(YARP\)](#) comme proxy inverse. Elle agit comme point d'entrée unique pour toutes les requêtes clientes et assure les responsabilités suivantes :

- **Routage** : Redirection des requêtes vers le microservice approprié selon l'URL.
- **Authentification** : Validation des tokens [JSON Web Token \(JWT\)](#), gestion des sessions sécurisées par cookies.
- **Événements** : Publication des événements de connexion/déconnexion vers [Kafka](#).
- **Sécurité** : Protection contre les attaques ([Cross-Site Scripting \(XSS\)](#), injections, etc.).

Architecture du Backend (Microservices .NET)

Le backend est composé de 7 microservices indépendants développés en **ASP.NET Core 10**, chacun responsable d'un domaine métier spécifique avec sa propre base de données **PostgreSQL 16**. Cette architecture garantit un couplage faible et une évolutivité indépendante de chaque service.

La communication entre microservices s'effectue via trois protocoles complémentaires :

- **REST** : Communication synchrone pour les opérations **Create, Read, Update, Delete (CRUD)** entre services.
- **gRPC** : Communication performante pour les échanges à faible latence (synchronisation utilisateur, récupération de détails).
- **Kafka** : Communication asynchrone et événementielle pour le découplage temporel entre services.

Le diagramme ci-dessous illustre les interactions entre les différents services.

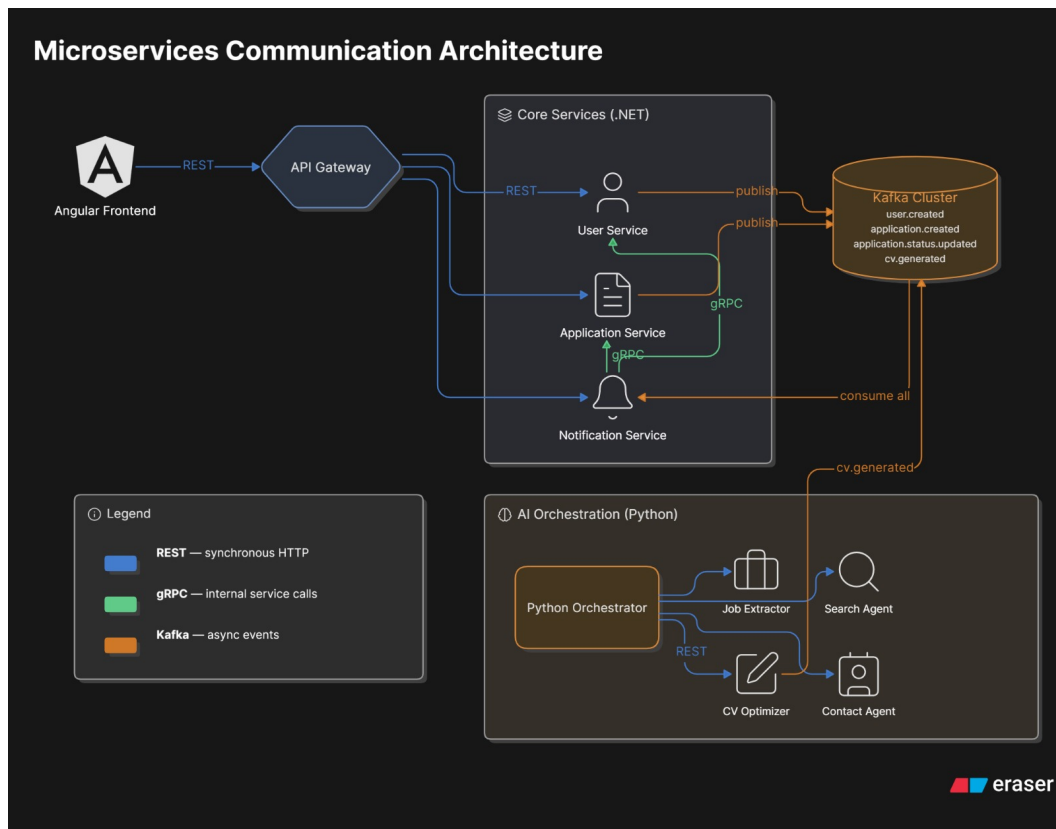


FIGURE 4.9 – Communication inter-services — **REST**, **gRPC** et **Kafka**

La structure globale du backend est présentée ci-après.

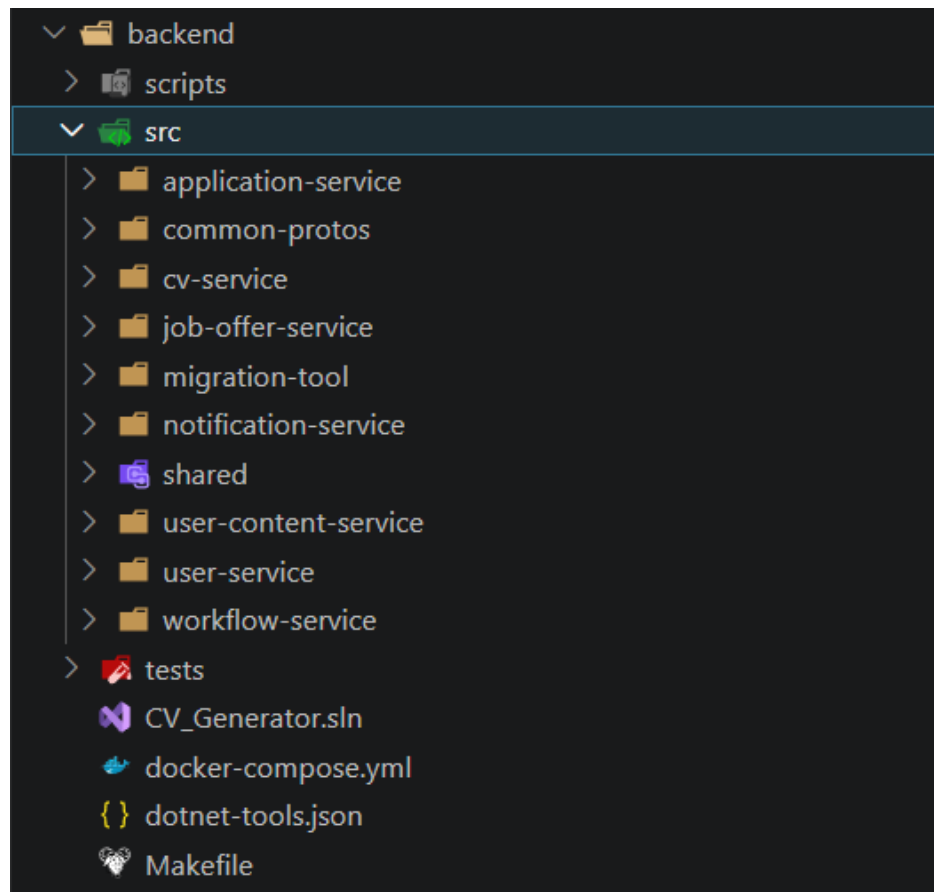


FIGURE 4.10 – Structure du projet Backend .NET

User Service Le **User Service** gère le cycle de vie des utilisateurs : inscription, authentification, mise à jour des profils et synchronisation avec **Keycloak**. Il expose une **gRPC** pour permettre aux autres services de récupérer les informations utilisateur. Il consomme les événements `user.registered` depuis **Kafka** pour créer le profil local.

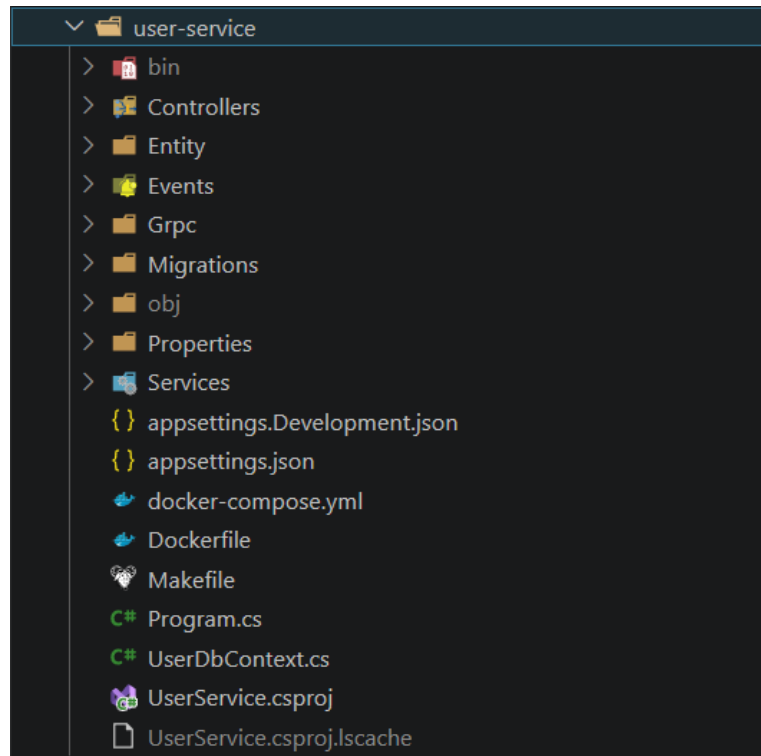


FIGURE 4.11 – Structure du User Service

User Content Service Le **User Content Service** gère le portfolio de l'utilisateur : projets, compétences, expériences professionnelles, formations, certifications, langues et réseaux sociaux. Il expose une [REST Application Programming Interface \(API\)](#) complète avec opérations [CRUD](#) et est consommé par le frontend ainsi que par le **CV Service** pour la génération de CV.

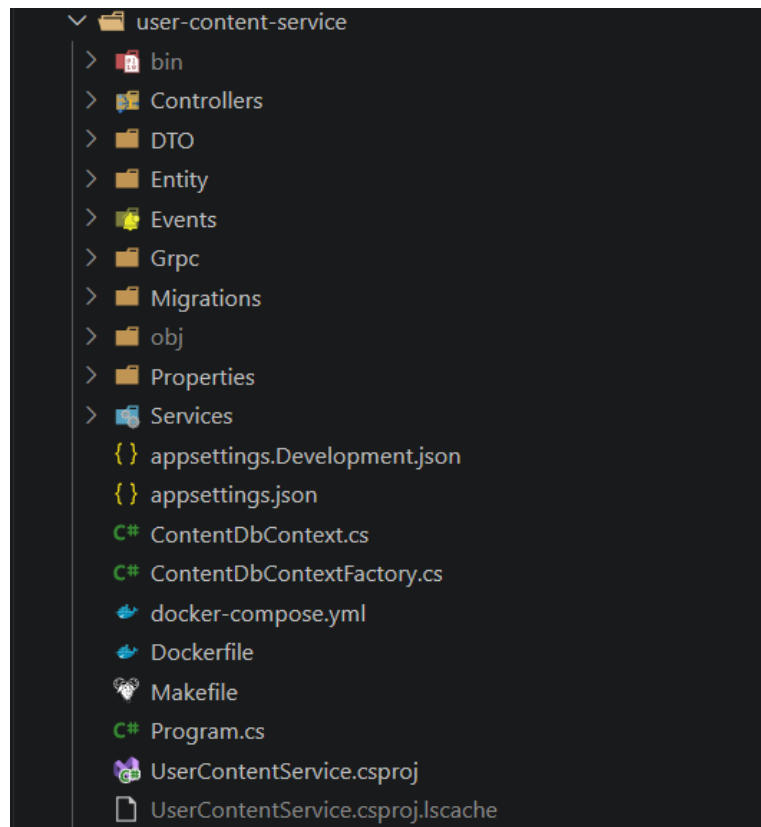


FIGURE 4.12 – Structure du User Content Service

CV Service Le **CV Service** est responsable de la création, du versionnement et de l'export des CV au format [Portable Document Format \(PDF\)](#). Il gère également les templates de CV et communique avec le **Workflow Service** pour déclencher les pipelines de génération. Les fichiers [PDF](#) générés sont stockés dans **MinIO**.

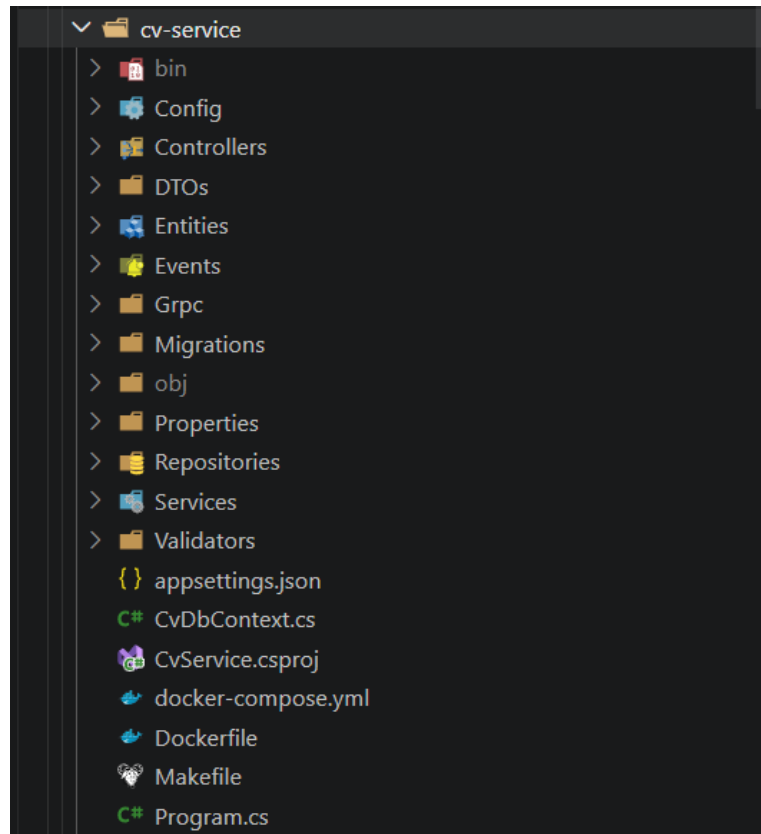


FIGURE 4.13 – Structure du CV Service

Workflow Service Le **Workflow Service** orchestre le pipeline de génération de CV. Il reçoit les demandes de génération, appelle séquentiellement les 6 agents IA via [HyperText Transfer Protocol \(HTTP\)](#), et publie les résultats sur [Kafka](#). Il utilise une base **PostgreSQL** avec l'extension **pgvector** pour le support des recherches vectorielles utilisées par l'agent de matching [RAG](#).

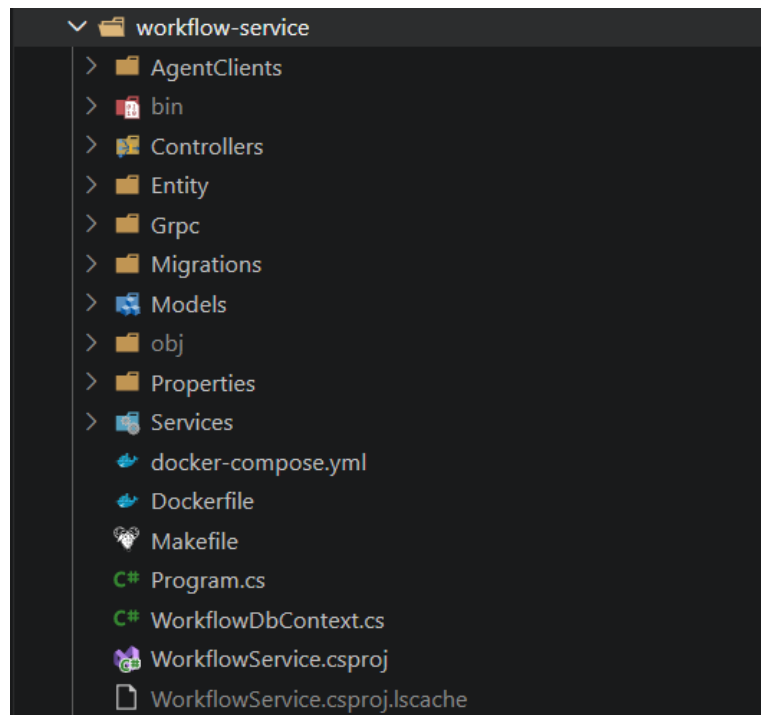


FIGURE 4.14 – Structure du Workflow Service

Application Service Le **Application Service** gère le suivi des candidatures. Il permet de créer, modifier et suivre l'état des candidatures (vue Kanban, liste, calendrier). Il communique avec le **User Service** via [gRPC](#) pour valider les utilisateurs et publie des événements de changement d'état sur [Kafka](#).

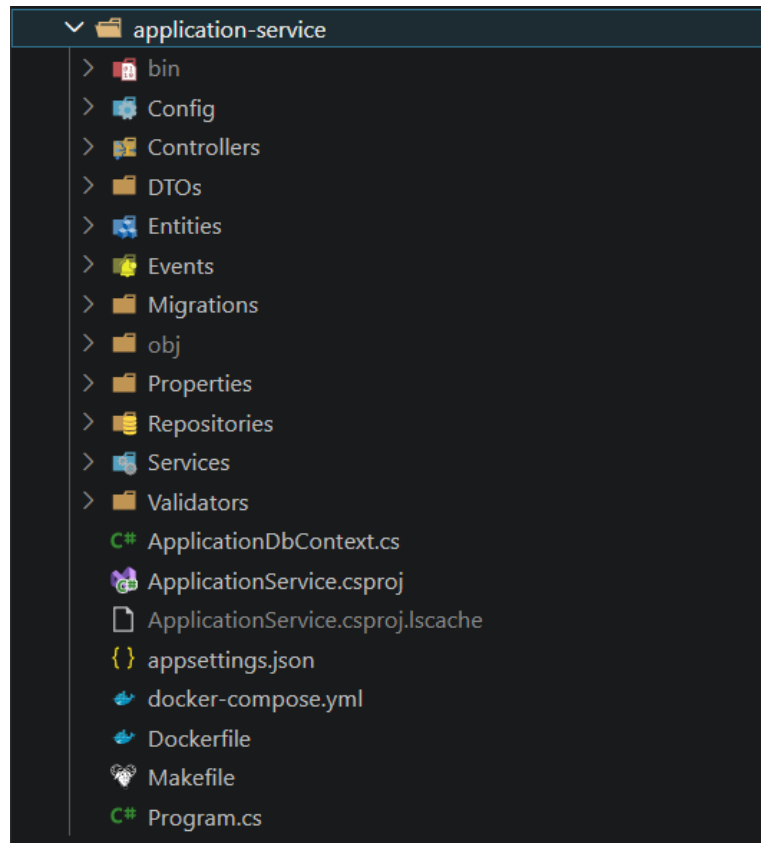


FIGURE 4.15 – Structure du Application Service

Job Offer Service Le **Job Offer Service** gère les offres d'emploi. Il expose des endpoints **REST** pour le **CRUD** des offres, déclenche le crawling automatique via un agent IA dédié, et utilise **SignalR** pour les notifications temps réel vers le frontend.

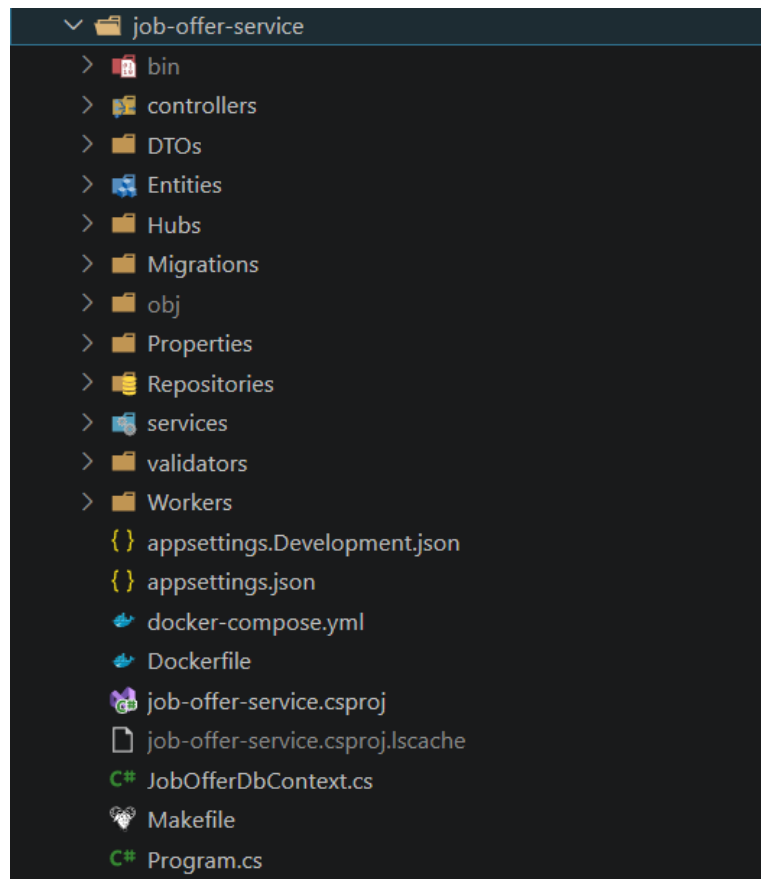


FIGURE 4.16 – Structure du Job Offer Service

Notification Service Le **Notification Service** est responsable de l'envoi de notifications. Il consomme les événements [Kafka](#), envoie des emails via **MailKit**, et gère les rappels planifiés avec **Hangfire**. Il assure la communication asynchrone avec les utilisateurs.

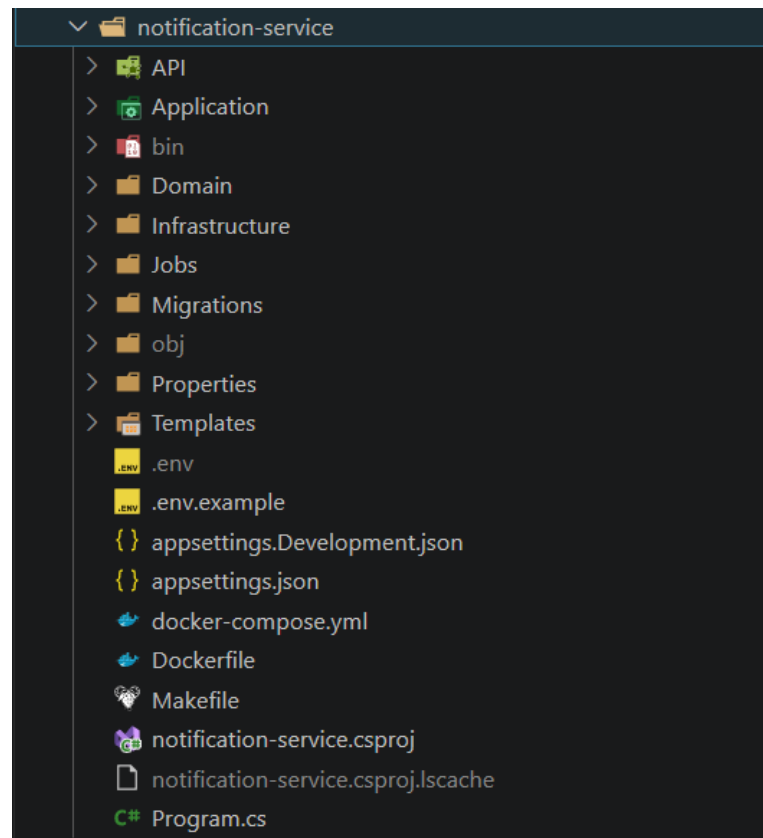


FIGURE 4.17 – Structure du Notification Service

Projets partagés En plus des 7 microservices, le backend inclut des projets partagés :

- **Common** : Bibliothèque de classes partagées (DTOs, modèles, constantes) utilisée par l'ensemble des services.
- **Common Protos** : Définitions des contrats [gRPC](#) (fichiers `.proto`) partagés entre les services.

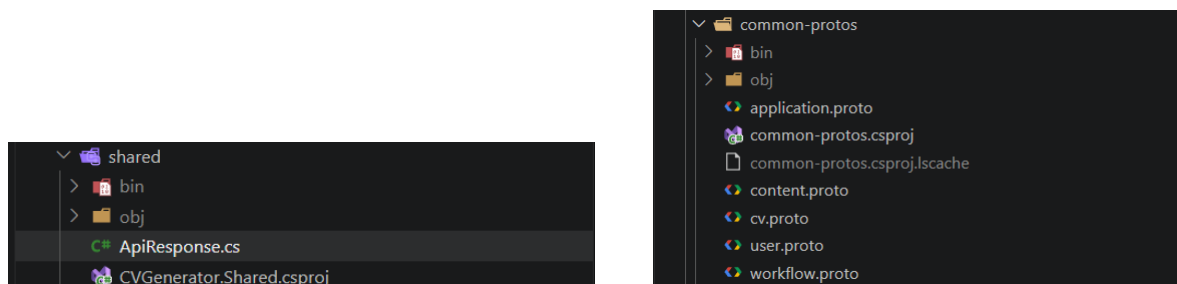


FIGURE 4.18 – Projets partagés — Common et Common Protos

Agents IA (Python)

Le pipeline d'agents IA est développé en **Python** avec **FastAPI**, **LangChain** et **LangGraph**. Il se compose de 6 agents spécialisés, chacun responsable d'une étape du processus de génération de CV :

- **Extractor** : Analyse l’offre d’emploi saisie par l’utilisateur et extrait les informations clés (compétences requises, expérience, formation).
- **Crawler** : Collecte automatiquement les offres d’emploi depuis des plateformes externes.
- **Search** : Recherche les meilleures correspondances dans le portfolio de l’utilisateur.
- **Optimizer** : Applique les techniques d’optimisation [Applicant Tracking System \(ATS\)](#) pour maximiser la compatibilité du CV avec les systèmes de recrutement.
- **Template** : Sélectionne et applique le template de CV le plus adapté au profil et au secteur visé.
- **Contact** : Génère une lettre de motivation personnalisée adaptée à l’offre ciblée.

L’orchestrateur, intégré au **Workflow Service**, coordonne l’exécution séquentielle de ces agents et gère le flux de données entre chaque étape.

Infrastructure transverse

L’infrastructure transverse regroupe les services communs à l’ensemble de l’application :

- **Kafka (mode KRaft)** : Système de messagerie asynchrone pour la communication événementielle. Il permet le découplage temporel entre les services et alimente le pipeline de génération de CV.
- **MinIO (Simple Storage Service (S3))** : Stockage d’objets compatible Amazon [S3](#) pour les fichiers [PDF](#) de CV générés et les templates.
- **Keycloak** : Serveur d’authentification centralisé supportant [Open Authorization \(OAuth\)](#) 2.0 et [OpenID Connect \(OIDC\)](#). Il gère les identités des utilisateurs et propose la fédération d’identités (Google, GitHub).
- **PostgreSQL 16** : 8 bases de données indépendantes (une par microservice + une pour Keycloak), chacune avec l’extension **pgvector** pour les recherches vectorielles.

4.2.3 Présentation des outils utilisés

Outils de développement

Le tableau suivant récapitule les principaux outils et technologies utilisés pour le développement :

TABLE 4.1 – Outils de développement

Couche	Technologies
Frontend	Angular 21, TypeScript, RxJS, Signals, SCSS, Vitest, Cypress
Backend	ASP.NET Core 10, Entity Framework Core, gRPC, YARP, Hangfire, MailKit
Agents IA	Python, FastAPI, LangChain, LangGraph, LangChain-Groq, Pydantic
Bases de données	PostgreSQL 16 avec extension pgvector
Authentification	Keycloak 26, OpenID Connect, JWT
Messagerie	Apache Kafka (KRaft mode)
Stockage	MinIO (API S3)
Conteneurisation	Docker, Docker Compose

Outils d’intégration et de déploiement continu (CI/CD)

TABLE 4.2 – Outils CI/CD et qualité

Outil	Rôle
GitHub Actions	Pipeline CI/CD (6 phases)
SonarCloud	Analyse statique de code et quality gates
Trivy	Scan de vulnérabilités (système de fichiers et images Docker)
OWASP ZAP	Tests de sécurité DAST (Dynamic Application Security Testing)
Docker	Conteneurisation multi-stage
GitHub Container Registry	Registre d’images Docker
DockerHub	Registre d’images secondaire

4.3 Cycle DevSecOps

4.3.1 Présentation et importance dans le projet

Le cycle DevSecOps intègre la sécurité à chaque étape du cycle de développement logiciel. Dans le cadre de **Propel**, cette approche est essentielle étant donné la nature sensible des données traitées (informations personnelles des utilisateurs, CV, données de connexion).

- L’adoption du cycle DevSecOps nous permet de :
- Détecter les vulnérabilités tôt dans le cycle de développement.

- Automatiser les contrôles de sécurité.
- Garantir la conformité aux bonnes pratiques.
- Assurer une traçabilité complète des modifications.

4.3.2 Outils DevSecOps utilisés

- **SonarCloud** : Analyse statique de code avec quality gates pour les projets Angular, .NET et Python. Des seuils de qualité (couverture de code, dette technique, duplication) doivent être respectés.
- **Trivy** : Scan de vulnérabilités sur le système de fichiers lors des phases statiques et sur les images Docker construites. Détection des dépendances obsolètes et des failles de sécurité connues.
- **OWASP ZAP** : Tests DAST (Dynamic Application Security Testing) en environnement d'intégration pour détecter les vulnérabilités applicatives (XSS, injection SQL, etc.).

4.3.3 Application du cycle DevSecOps dans le projet

Le cycle DevSecOps est appliqué à travers le pipeline CI/CD en six phases :

1. **Phase 1 - Static Checks** : Validation des fichiers Docker Compose, scan Trivy filesystem, analyse SonarCloud pour le frontend, le backend et les agents IA.

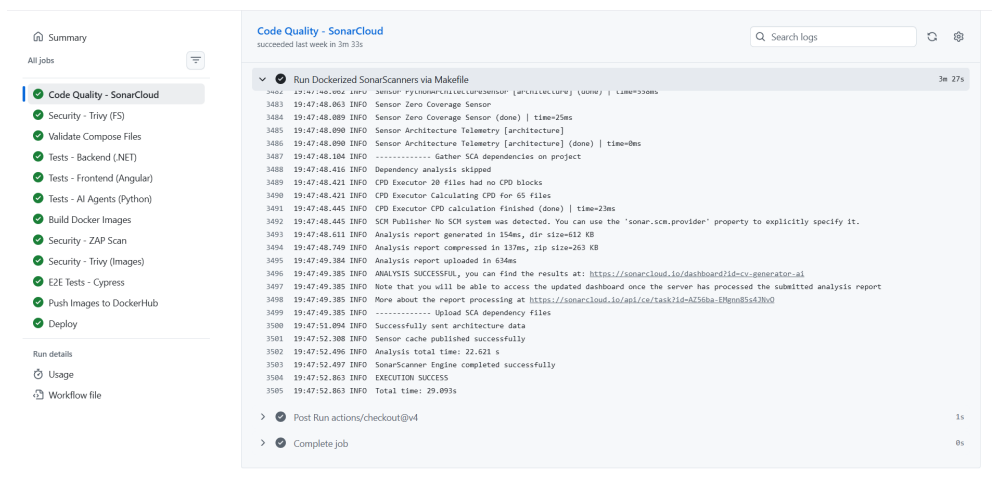


FIGURE 4.19 – Analyse SonarCloud réussie pour le projet

2. **Phase 2 - Component Tests** : Exécution des tests unitaires (.NET avec xUnit, Angular avec Vitest, Python avec pytest) et génération des rapports de couverture.
3. **Phase 3 - Integration / E2E** : Tests d'intégration et tests E2E avec Cypress, suivis d'un scan de sécurité OWASP ZAP.

4. **Phase 4 - Build & Scan** : Construction des images Docker multi-stage, suivie d'un scan Trivy des images.
5. **Phase 5 - Registry Push** : Publication des images validées vers GitHub Container Registry et DockerHub.

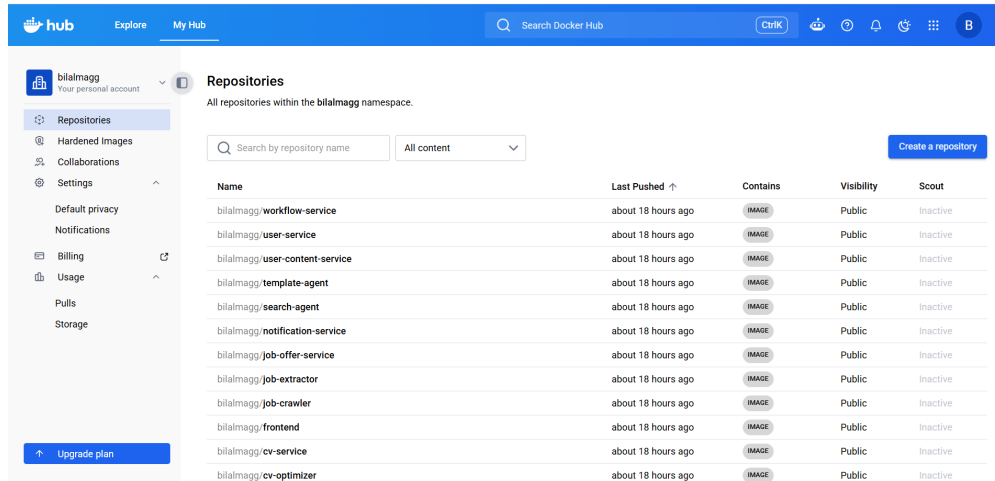


FIGURE 4.20 – Images des services publiées sur DockerHub après succès du pipeline

6. **Phase 6 - Deploy** : Déploiement de l'application sur l'environnement cible.

4.3.4 Sécurité applicative

La sécurité applicative est assurée à plusieurs niveaux :

- **Authentification centralisée** : Keycloak avec OpenID Connect, support OAuth2 (Google, GitHub), gestion des sessions par cookies sécurisés JWT.
- **Protection des API** : Validation des tokens JWT, gestion des rôles et permissions.
- **Sécurisation des communications** : HTTPS, chiffrement des données sensibles.
- **Scan de vulnérabilités** : Automatisé via Trivy (images et dépendances) et OWASP ZAP (tests dynamiques).
- **Gestion des secrets** : Fichier `.env` avec variables d'environnement, exclusion des secrets du versionnement via `.gitignore`.

4.4 Gestion du code source

4.4.1 Utilisation de Git

Le code source du projet est versionné avec **Git** et hébergé sur **GitHub**. L'utilisation de Git permet un suivi précis de l'historique des modifications, une collaboration efficace entre les membres de l'équipe et une intégration fluide avec le pipeline CI/CD.

4.4.2 Organisation des branches

La stratégie de branches adoptée est une approche **Git Flow** simplifiée :

- **main** : Branche de production, protégée, ne reçoit que des merges validés.
- **develop** : Branche d'intégration continue pour le développement.
- **feature/*** : Branches de fonctionnalités créées à partir de **develop** et fusionnées via Pull Request.
- **hotfix/*** : Branches de correction urgente créées à partir de **main**.

FIGURE 4.21 – Organisation des branches Git

4.4.3 Gestion des commits et collaboration

La collaboration s'effectue via **Pull Requests** sur GitHub. Chaque PR est soumise à une revue de code par le chef d'équipe (Bilal Magguilej) avant d'être fusionnée. Le pipeline CI/CD s'exécute automatiquement sur chaque PR pour valider la qualité et la sécurité du code proposé.

4.5 Implémentation

4.5.1 Développement Backend

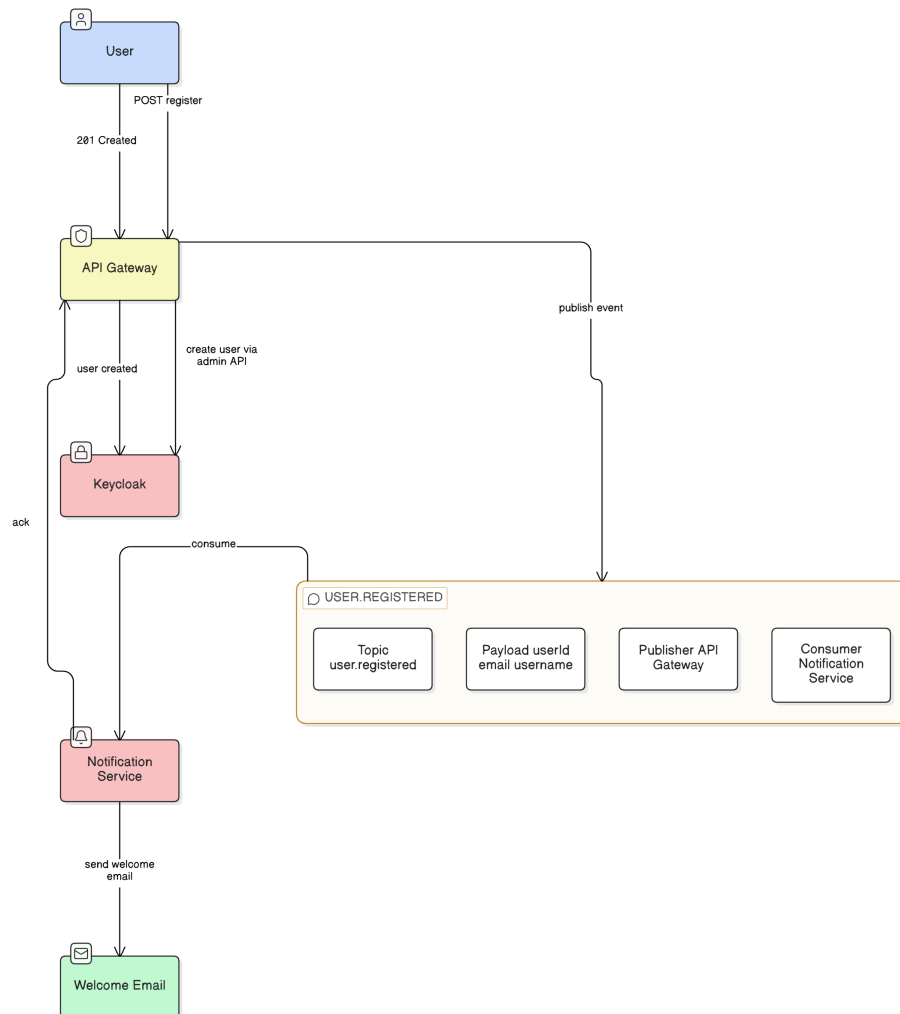
Le backend est composé de 7 microservices .NET, chacun responsable d'un domaine métier spécifique :

- **User Service** : Gestion des utilisateurs, synchronisation avec Keycloak, communication gRPC.
- **User Content Service** : Gestion du portfolio (projets, compétences, expériences, formations, certifications, langues).
- **CV Service** : Création et versionnement des CV, gestion des templates, export PDF.
- **Workflow Service** : Orchestration de la génération de CV, appel aux agents IA.
- **Application Service** : Suivi des candidatures, communication gRPC avec User Service.
- **Job Offer Service** : Gestion des offres d'emploi, déclenchement du crawling, hub SignalR.
- **Notification Service** : Envoi d'emails (MailKit), rappels planifiés (Hangfire), consommateur Kafka.

La passerelle API (**YARP**) est un service .NET dédié qui agit comme point d'entrée unique. Elle gère l'authentification, le routage vers les microservices et la publication

d'événements Kafka.

La communication événementielle est assurée par **Apache Kafka** en mode KRaft. Le diagramme ci-dessous illustre les principaux événements échangés entre les services via Kafka.



eraser

FIGURE 4.22 – Flux des événements Kafka — Publications et consommations

Les topics Kafka utilisés dans le projet sont visibles dans la capture ci-dessous.

Name	Partitions	% Preferred	# Under-replicated	Custom Config
interest-created	1	100 %	0	No
language-created	1	100 %	0	No
language-updated	1	100 %	0	No
project-created	1	100 %	0	No
project-updated	1	100 %	0	No
skill-created	1	100 %	0	No
skill-updated	1	100 %	0	No
social-link-created	1	100 %	0	No

FIGURE 4.23 – Topics Kafka — user.registered, user.created, trigger-live-crawl

4.5.2 Pipeline de génération de CV

Le **Workflow Service** orchestre le pipeline de génération de CV, composé de 5 agents IA agissant en séquence. Chaque agent traite une étape spécifique du flux de travail.

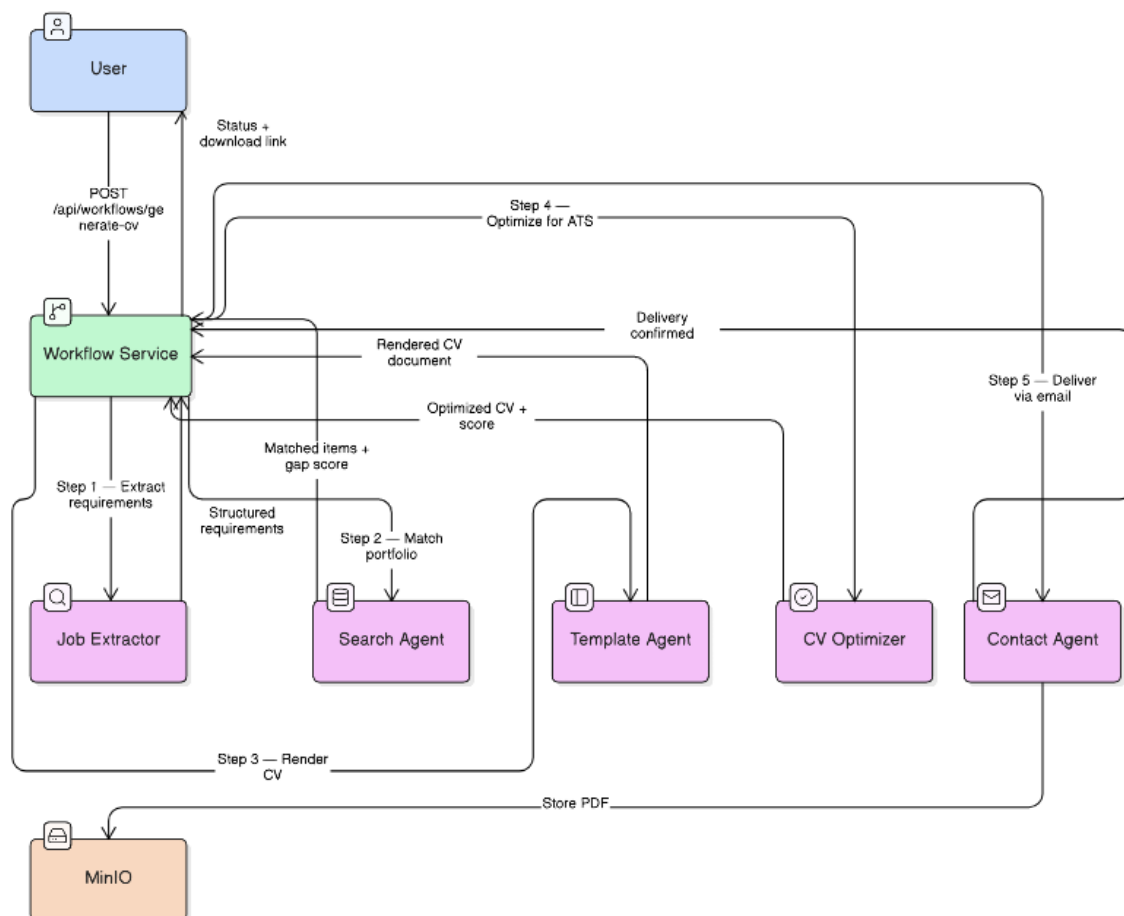


FIGURE 4.24 – Pipeline de génération de CV — De l’offre d’emploi à l’envoi par email

4.5.3 Développement Frontend

Le frontend est développé avec **Angular 21** en utilisant les **standalone components**, **RxJS** pour la gestion asynchrone et les **Signals** pour la réactivité. L'interface suit le design system **Propel** avec des couleurs en espace OKLCH.

Les principales fonctionnalités du frontend incluent :

- Pages de connexion et d'inscription (Keycloak).
- Tableau de bord avec statistiques et graphiques.
- Interface de génération de CV (saisie d'offre d'emploi).
- Vue Kanban pour le suivi des candidatures (drag-and-drop).
- Vue calendrier avec rappels.
- Gestion complète du portfolio.
- Vues liste et analytique des candidatures.

4.5.4 Base de données

L'architecture microservices implique l'utilisation de 8 bases de données **PostgreSQL 16** indépendantes, une par microservice, plus une pour Keycloak. Chaque base utilise l'extension **pgvector** pour le support des recherches vectorielles utilisées par l'agent de matching RAG.

4.5.5 Documentation des API

Chaque microservice expose ses endpoints via une interface **Swagger UI** intégrée, facilitant le test et la documentation des API. Les captures ci-dessous présentent la documentation de chaque service.

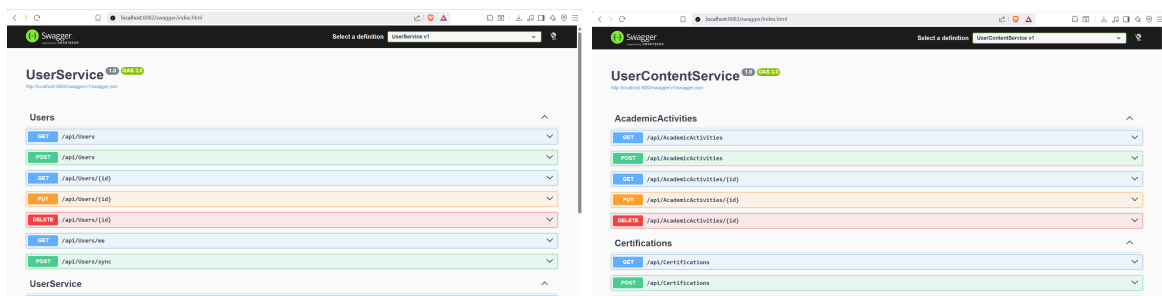


FIGURE 4.25 – Documentation Swagger — User Service et User Content Service

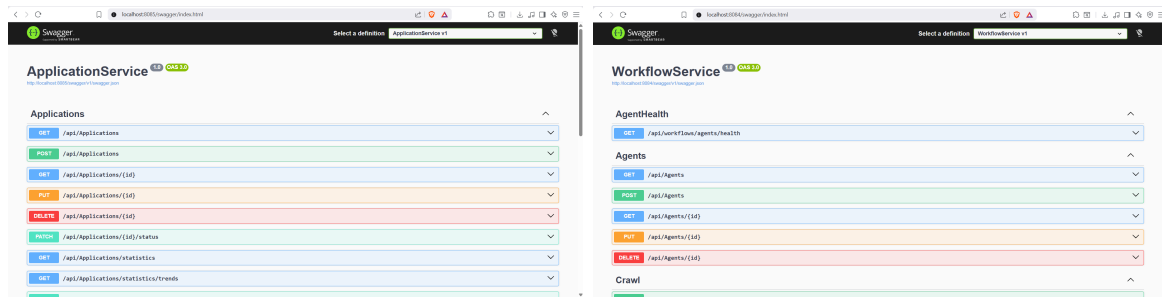


FIGURE 4.26 – Documentation Swagger — Application Service et Workflow Service

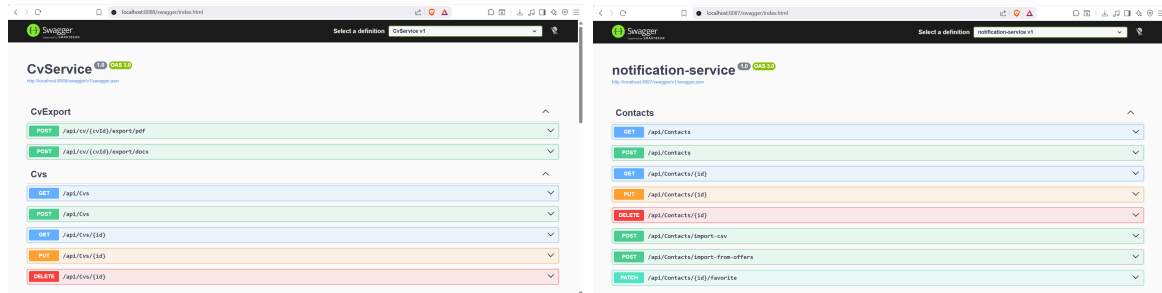


FIGURE 4.27 – Documentation Swagger — CV Service et Notification Service

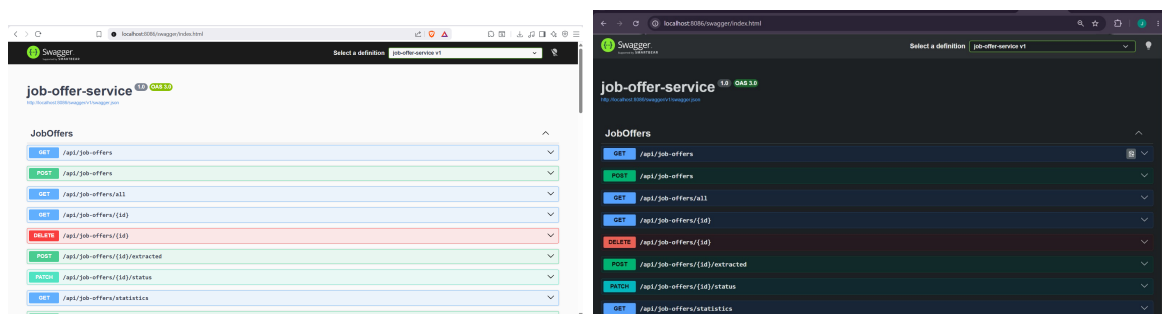


FIGURE 4.28 – Documentation Swagger — Job Offer Service

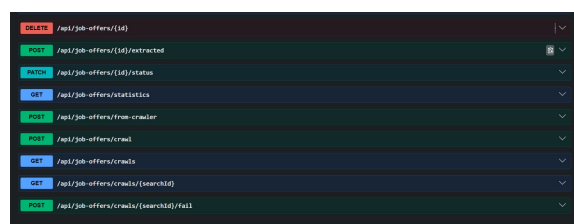


FIGURE 4.29 – Documentation Swagger — Job Offer Service (suite)

4.6 Déploiement

4.6.1 Mise en place de l'environnement de développement

L'environnement de développement est entièrement conteneurisé. Un fichier Makefile à la racine du projet automatise les commandes courantes (démarrage des services, exé-

cution des migrations, lancement des tests). Chaque sous-projet (backend, ai_agents) possède également son propre Makefile.

4.6.2 Dockerisation de l'application

L'ensemble de l'application est dockerisé avec des **Dockerfiles multi-stage** pour optimiser la taille des images de production. Au total, 18 Dockerfiles ont été créés couvrant l'ensemble des services.

Les images sont construites en deux étapes :

1. **Build** : Compilation et génération des artefacts.
2. **Runtime** : Image légère ne contenant que l'exécutable et ses dépendances.

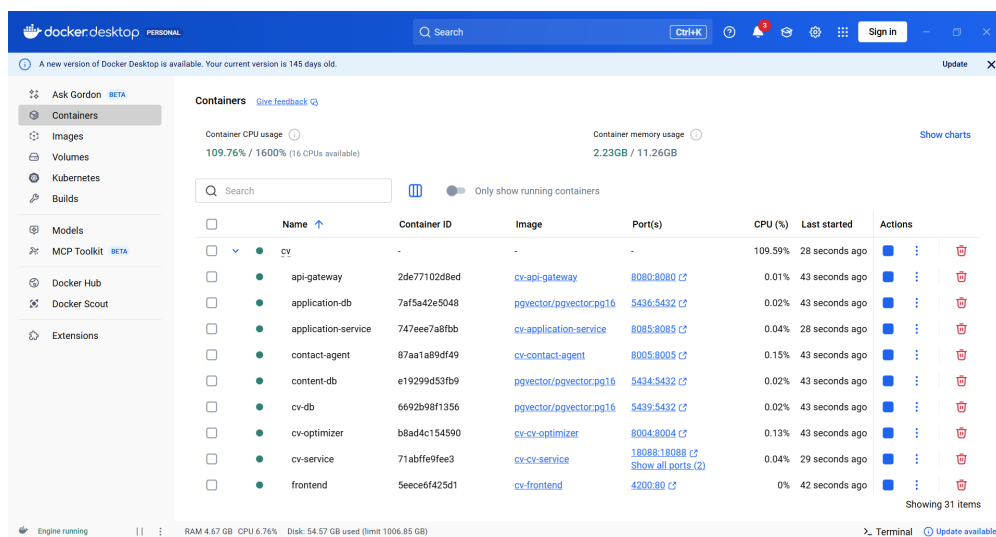


FIGURE 4.30 – Conteneurs Docker en cours d'exécution

4.6.3 Pipeline CI/CD

Le pipeline CI/CD est implémenté avec **GitHub Actions** et comprend 6 phases séquentielles. Chaque phase doit réussir pour passer à la suivante.

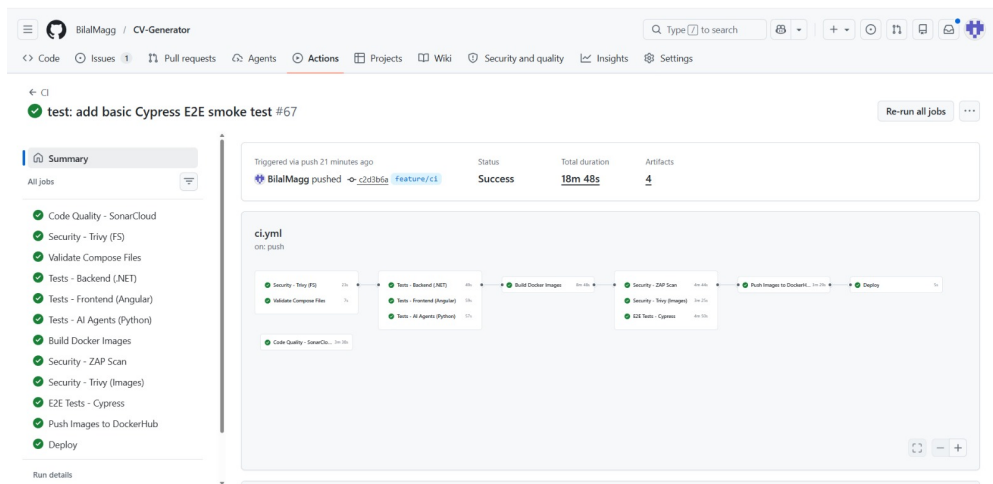


FIGURE 4.31 – Pipeline CI/CD — GitHub Actions (succès)

4.7 Présentation de l'application

4.7.1 Interface utilisateur

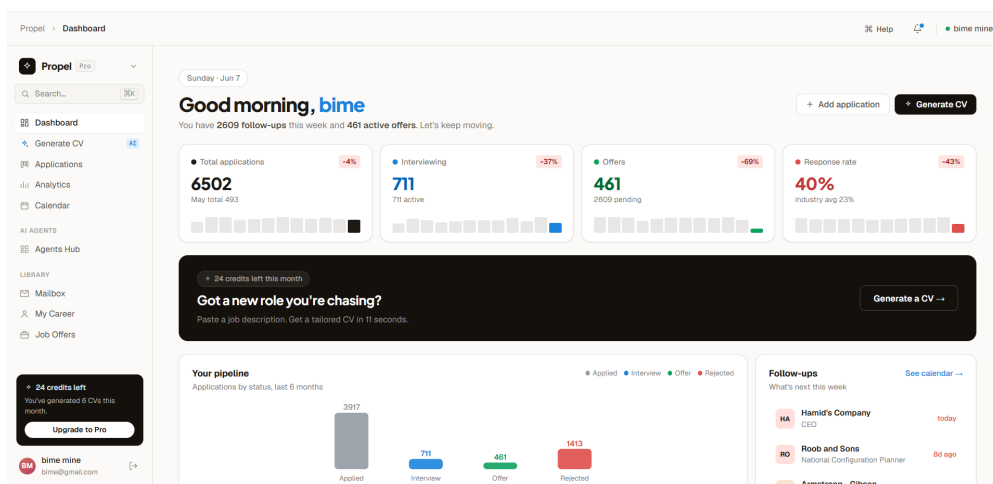


FIGURE 4.32 – Tableau de bord principal

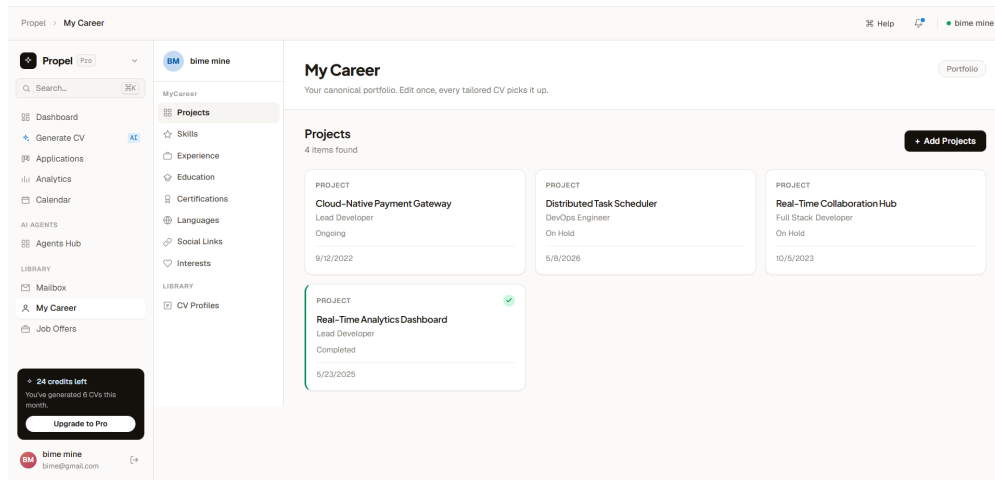


FIGURE 4.33 – Interface de gestion du portfolio et des compétences

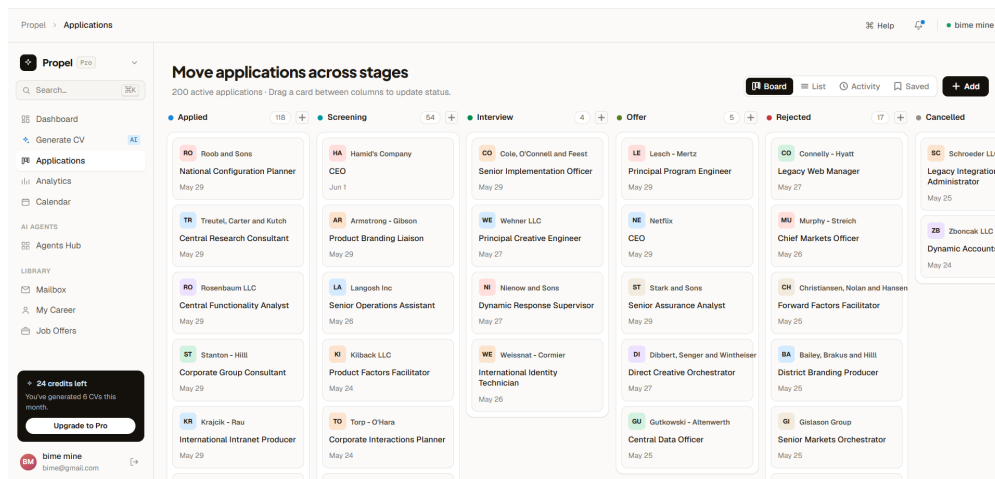


FIGURE 4.34 – Tableau Kanban de suivi des candidatures

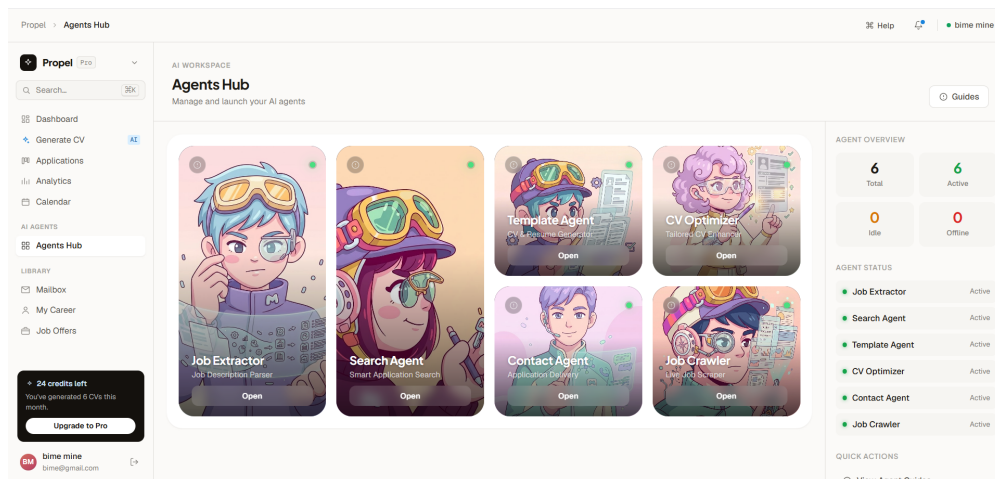


FIGURE 4.35 – Interface des agents IA pour la génération de CV

4.7.2 Fonctionnalités principales

Le système **Propel** offre les fonctionnalités principales suivantes :

- **Génération intelligente de CV** : Pipeline de 6 agents IA (extraction, matching RAG, template, optimisation ATS, contact) coordonnés par un orchestrateur.
- **Suivi de candidatures** : Vue Kanban avec drag-and-drop, vue liste, vue calendrier, graphiques analytiques.
- **Gestion de portfolio** : CRUD complet pour projets, compétences, expériences, formations, certifications, langues et réseaux sociaux.
- **Crawling d'offres d'emploi** : Agent IA dédié qui collecte automatiquement les offres depuis des plateformes externes.
- **Notifications temps réel** : SignalR pour les mises à jour en direct, Hangfire pour les rappels planifiés, emails via MailKit.

4.7.3 Démonstrations du fonctionnement

Cette section présente une démonstration des principales interfaces de l'application **Propel**.

Authentification et page d'accueil

L'application propose une page d'accueil présentant les fonctionnalités clés, accessible sans authentification. La connexion s'effectue via un formulaire sécurisé ou via les fournisseurs OAuth2 (Google, GitHub).

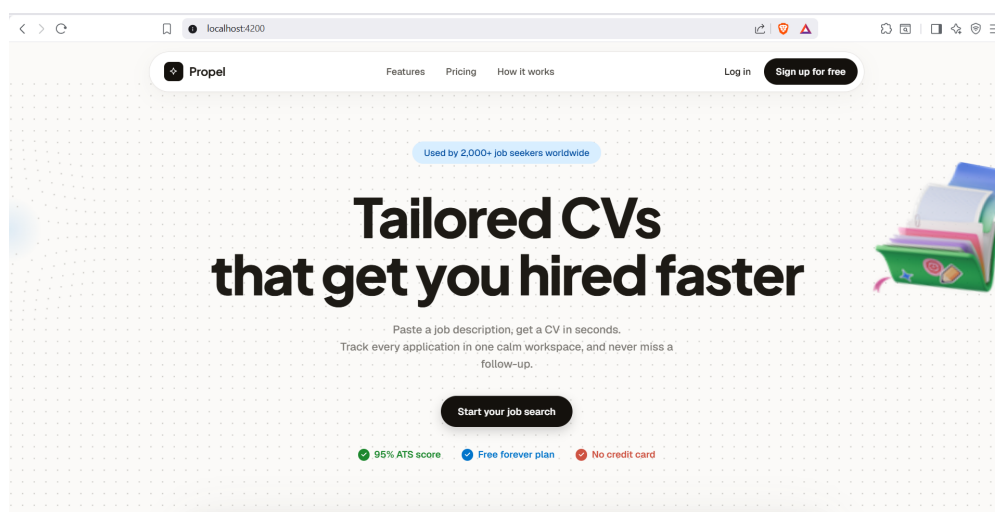


FIGURE 4.36 – Page d'accueil de **Propel**

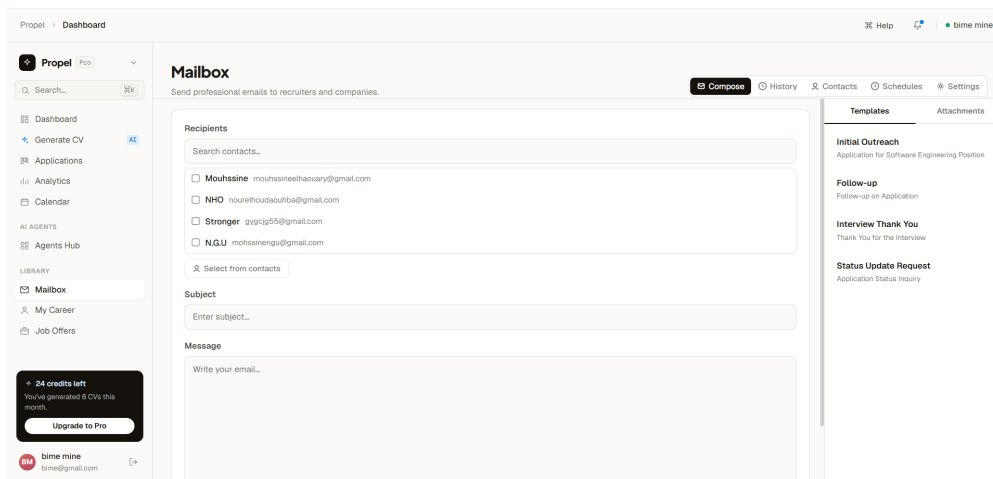


FIGURE 4.39 – Boîte de réception des notifications

4.8 Conclusion

Ce chapitre a présenté en détail la réalisation technique du projet **Propel**. Nous avons couvert l'architecture physique et logicielle, les outils utilisés, le cycle DevSecOps, la gestion du code source, l'implémentation des différentes couches et le déploiement. Le projet démontre une maîtrise complète de la chaîne DevOps, de l'écriture du code jusqu'au déploiement automatisé en passant par la sécurité et la qualité.

Chapitre 5

Conclusion générale et perspectives

5.1 Conclusion générale

Le projet **Propel** nous a permis de concevoir et réaliser une plateforme complète de génération de CV assistée par intelligence artificielle. Ce projet d'intégration a mobilisé l'ensemble des compétences acquises durant notre formation, du développement web à l'architecture microservices, en passant par la conteneurisation, l'intégration continue, la sécurité applicative et l'observabilité.

L'approche DevOps adoptée, avec une organisation en microservices et un pipeline CI/CD complet intégrant qualité et sécurité, a permis de livrer une application robuste et industrialisée. La pipeline d'agents IA basée sur LangChain et LangGraph démontre la capacité à intégrer des technologies avancées d'intelligence artificielle dans une application concrète.

Sur le plan méthodologique, l'adoption de SCRUM avec des réunions quotidiennes et une communication continue via Google Meets et WhatsApp a favorisé une collaboration efficace au sein de l'équipe de six développeurs fullstack, chacun apportant son expertise dans son domaine de responsabilité secondaire.

5.2 Perspectives

Plusieurs perspectives d'amélioration et d'évolution sont envisageables pour **Propel** :

- **Déploiement cloud** : Migrer l'infrastructure vers un fournisseur cloud (AWS, Azure, GCP) avec orchestration Kubernetes pour une meilleure scalabilité.
- **Modèles de langage additionnels** : Intégrer d'autres fournisseurs de LLM (OpenAI, Anthropic, modèles open source en local) pour diversifier les capacités de génération.
- **Internationalisation** : Ajouter le support multilingue pour l'interface et la

génération de CV dans différentes langues.

- **Analyse avancée :** Développer des fonctionnalités d'analyse prédictive pour suggérer les offres d'emploi les plus pertinentes en fonction du profil de l'utilisateur.
- **Mode hors ligne :** Proposer une version allégée de l'application fonctionnant sans connexion Internet pour la consultation des CV et candidatures.
- **Place de marché :** Créer une plateforme de mise en relation entre candidats et recruteurs utilisant les données de profiling générées par l'application.

Bibliographie

Angular documentation, 2025. URL <https://angular.dev>.

Apache kafka documentation, 2025. URL <https://kafka.apache.org/documentation>.

Asp.net core documentation, 2025. URL <https://learn.microsoft.com/en-us/aspnet/core>.

Cypress documentation, 2025. URL <https://docs.cypress.io>.

Docker documentation, 2025. URL <https://docs.docker.com>.

Fastapi documentation, 2025. URL <https://fastapi.tiangolo.com>.

Grafana documentation, 2025. URL <https://grafana.com/docs>.

k6 documentation, 2025. URL <https://grafana.com/docs/k6>.

Keycloak documentation, 2025. URL <https://www.keycloak.org/documentation>.

Langchain documentation, 2025. URL <https://python.langchain.com>.

Postgresql documentation, 2025. URL <https://www.postgresql.org/docs>.

Prometheus documentation, 2025. URL <https://prometheus.io/docs>.

Sonarcloud documentation, 2025. URL <https://docs.sonarcloud.io>.

Terraform documentation, 2025. URL <https://developer.hashicorp.com/terraform/docs>.

Trivy documentation, 2025. URL <https://trivy.dev>.